



OPEN DMSTG-AD: an SDN intrusion detection method based on dynamic multi-scale spatio-temporal graph neural network

Ji Zhao, Damin Zhang✉, Qing He, Mu Lin & Yuhang Yang

Software-defined networking (SDN) centralizes control, simplifying network management and configuration while simultaneously increasing vulnerability to attacks, such as large-scale Distributed Denial-of-Service (DDoS) attacks or network topology spoofing. Conventional intrusion detection methods that rely on feature engineering or static graph modeling often fail to capture the complex topological dependencies and dynamic temporal patterns inherent in network traffic. To address this issue, a dynamic multi-scale spatio-temporal graph neural network framework, named DMSTG-AD, is proposed for intrusion detection. The model employs Gated Recurrent Unit (GRU)-driven dynamic node embeddings, an adaptive adjacency matrix, and edge-node collaborative convolution to extract spatial dependencies, while multi-scale dilated convolutions and a bidirectional GRU jointly capture short-term fluctuations and long-term trends. A spatio-temporal cross-attention mechanism is designed to integrate spatial and temporal features, thereby enhancing anomaly detection capabilities. Experimental results demonstrate that the proposed DMSTG-AD model achieves a multi-classification accuracy of 99.34% on the CIC-IDS2017 dataset and an overall accuracy of 99.88% on the InSDN dataset, both of which significantly exceed those of existing mainstream methods. Ablation experiments further validate the critical importance of dynamic modeling, dual-channel feature extraction, and the cross-attention mechanism in improving detection performance. This study not only offers a novel approach for high-precision intrusion detection in SDN environments but also broadens the application prospects of dynamic graph neural networks in the field of network security.

Keywords Graph neural network, Software-defined networking, Network intrusion detection system, Dynamic spatio-temporal modeling

Traditional IP networks typically employ a distributed control architecture, in which each network device (e.g., routers and switches) simultaneously maintains its own control plane and data plane. This design necessitates the manual configuration of policies on a per-device basis, which not only increases the operational burden but also renders the system prone to errors¹. To address these challenges, SDN has been proposed. SDN decouples the control plane from the data plane, centralizing control logic within one or more controllers while delegating data forwarding to the switching infrastructure. During the forwarding process, the controller issues unified instructions to the switching devices via the southbound interface, thereby determining the packet forwarding paths. This centralized control paradigm enables policies to be configured and managed uniformly from the controller side, significantly enhancing network flexibility and reducing management complexity².

However, this centralization and programmability also introduce new security risks, rendering SDN more susceptible to various network intrusion attacks³. Specifically, each plane faces distinct attack patterns. In the control plane, topology poisoning attacks consume controller resources and distort its perception of the network topology, thereby disrupting global decision-making. In the data plane, Denial-of-Service (DoS) attacks overwhelm bandwidth and flow table resources with massive malicious requests, preventing legitimate traffic from being processed efficiently⁴. In the application plane, attacks may lead to service interruptions, abnormal termination of applications, and failures in controller–application communications⁵. Consequently, developing efficient Network Intrusion Detection Systems (NIDS) tailored specifically for SDN environments is of critical importance.

College of Big Data and Information Engineering, Guizhou University, Guiyang 550025, China. ✉email: dmzhang@gzu.edu.cn

Existing NIDS approaches can be broadly categorized into two types: signature-based and behavior-based methods. The former relies on predefined rules, features, and thresholds to classify traffic, which makes it difficult to cope with rapidly evolving and highly diverse attack patterns⁶. The latter leverages machine learning to learn and generalize traffic patterns, enabling better detection of complex, unknown, or variant attacks⁷. In traditional machine learning, methods such as Support Vector Machines (SVM) and Random Forests (RF) have been widely applied to NIDS⁸. For example, Gu et al.⁹ proposed an SVM-based intrusion detection framework that achieved high accuracy and robustness across multiple datasets. Similarly, Ahuja et al.¹⁰ designed the SVC-RF model based on RF for detecting DDoS attacks in SDN, achieving high accuracy with low false alarm rates on experimental datasets. Although these methods are effective, their performance is heavily dependent on feature engineering, which limits transferability and scalability. With the advent of deep learning (DL), models such as Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNN) have been introduced into NIDS to automatically learn more discriminative representations. For instance, Wu et al.¹¹ employed CNNs to reduce false positives and improve minority-class detection, while Malik et al.¹² combined CNNs and Long Short-Term Memory (LSTM) networks to jointly capture spatial and temporal features, thereby enhancing detection performance in SDN scenarios.

It is worth noting that both traditional machine learning and early DL approaches often overlook network topology, despite its critical role in revealing attack propagation patterns and inter-component interactions. Recently, Graph Neural Networks (GNNs) have introduced a new paradigm for NIDS by enabling direct modeling of spatial dependencies at the node, edge, and subgraph levels¹³. For example, Caville et al.¹⁴ proposed Anomal-E, a self-supervised framework that represents network traffic as graphs and applies E-GraphSAGE to encode edge features for effective detection without the need for labeled data. Altaf et al.¹⁵ proposed NE-GConv, which jointly extracts node and edge features, achieving improvements in both accuracy and false positive rates. These studies collectively highlight the advantages of graph-based modeling for intrusion detection in networked environments.

Nevertheless, existing GNN-based NIDS approaches still face three major limitations: (1) Limited granularity in temporal modeling. Most methods perform temporal modeling along a single path, either at the node or graph level, and often fail to simultaneously capture both local and global dynamics. (2) Decoupling of spatial and temporal dimensions. Many approaches emphasize either spatial topology or temporal evolution, yet struggle to achieve genuine spatio-temporal fusion within a unified representation space. (3) Insufficient modeling of dynamic graphs. Real-world network traffic is inherently dynamic; however, many methods approximate it as static graphs, which hinders the capture of time-varying characteristics in topology and inter-node interactions.

To overcome these limitations, this paper introduces a Dynamic Multi-Scale Spatio-temporal Cross-Attention Graph Neural Network for Anomaly Detection (DMSTG-AD). The proposed framework leverages dynamic node embeddings and adaptive adjacency matrices to model temporal changes in both node semantics and network topology. Furthermore, it integrates multi-scale dilated Temporal Convolutional Networks (TCN) with Bidirectional Gated Recurrent Units (BiGRU) to effectively capture both short-term traffic bursts and long-term temporal dependencies. On the spatial side, edge-node collaborative convolution is employed to extract both local and global structural dependencies. Finally, a Spatio-temporal Cross-Attention (STCA) mechanism fuses spatial and temporal representations at the node level, generating robust spatio-temporal features for anomaly detection. The main contributions of this paper are summarized as follows:

1. Network traffic is formulated as a time-evolving graph sequence, and the DMSTG-AD model is proposed. The model integrates Graph Convolutional Networks (GCN), TCN, and BiGRU to extract spatio-temporal features. These features are fused at the node level via a STCA mechanism, enabling fine-grained anomaly scoring and classification while jointly capturing topological structure and temporal dynamics.
2. A GRU-driven dynamic node embedding module captures short-term behavioral variations at the node level, while multi-scale dilated TCN and BiGRU jointly model graph-level temporal dependencies, balancing local bursts and global trends. To the best of the authors' knowledge, few existing works in SDN intrusion detection simultaneously consider both node-level and graph-level temporal modeling within a unified framework, as achieved by the proposed method.
3. Experiments conducted on two public datasets, CIC-IDS2017 and InSDN, demonstrate that DMSTG-AD significantly outperforms existing methods in both binary and multi-class classification tasks in terms of accuracy and F1 score. These results validate the effectiveness of dynamic modeling, dual-channel spatio-temporal feature extraction, and cross-attention fusion.

The remainder of this paper is organized as follows: Section “[Related work](#)” reviews related work, Section “[Methodology](#)” introduces the proposed NIDS method, Section “[Experiments](#)” reports extensive experimental results, and Section “[Conclusion](#)” concludes the paper and discusses future directions.

Related work

Deep learning-based NIDS

In recent years, DL has gained considerable attention in the field of intrusion detection due to its strong representational capacity and automatic feature learning capabilities. DL excels at learning hierarchical representations from large-scale, high-dimensional network data, thereby improving the characterization of complex and unknown attacks. As a result, DL-based NIDS have demonstrated superior adaptability to diverse attack patterns¹⁶.

For instance, ElSayed et al.¹⁷ proposed a hybrid DL model for SDN based on CNN and introduced the SD-Reg regularization technique to mitigate overfitting, thereby improving the detection of unknown attacks. Ravi et al.¹⁸ employed Kernel Principal Component Analysis (KPCA) to select optimal features, which were then

fed into an ensemble meta-classifier, achieving 99% detection accuracy for network attacks and 97% overall classification accuracy. To improve detection and response speed, Assis et al.¹⁹ developed an SDN defense system based on single-IP flow record analysis, employing GRU to detect DDoS and intrusion attacks. The system achieved faster mitigation responses by enabling direct flow inspection, thereby reducing the impact of attacks on SDN. Elsayed et al.²⁰ designed a two-level Security-Aware Intrusion Detection System (SATIDS) based on an improved LSTM network for SDN protection, significantly enhancing accuracy, recall, and precision. Tri et al.²¹ proposed DeepMonitor, an SDN-based traffic monitoring framework, and developed a Double Deep Q-Network (DDQN) algorithm alongside a federated DDQN-based monitoring mechanism. This approach effectively alleviated flow table overflow at edge nodes, increased the number of matching fields, and improved NIDS performance for DDoS detection. To more precisely identify DDoS attacks in SDN, Wang et al.²² designed DDosTC, a hybrid detection mechanism combining Transformer and CNNs. The method outperformed state-of-the-art models in average accuracy, recall, and F1-score, and improved the average AUC by 2.52%. Furthermore, Behera et al.²³ proposed an ensemble model integrating CNN, Bi-GRU, and Bidirectional LSTM (Bi-LSTM). Experimental results demonstrated that this model achieved high detection accuracy, low detection latency, and reduced resource consumption.

In summary, DL-based approaches can automatically learn discriminative features and deliver promising results across multiple attack scenarios. However, most existing DL methods rely on the assumptions of independent samples and flattened representations, which hinders their ability to explicitly capture the structural patterns and propagation paths of attacks within the network topology. This limitation motivates the integration of graph-based representations and graph learning techniques into intrusion detection.

GNN-based NIDS

Although DL models have achieved remarkable progress in network attack detection, vector- or sequence-based “flat” representations fail to capture the structural behaviors and topological dependencies associated with attacks—features that are essential for effective detection and attribution. In contrast, representing network traffic as graphs offers a more robust and abstract representation of the network, making it more difficult for attackers to evade detection through localized perturbations. The emergence of GNNs has introduced a novel modeling paradigm for NIDS. Based on how they treat temporal and topological dimensions, existing GNN-based NIDS can be broadly categorized into three groups, each designed to address a specific aspect yet suffering from inherent limitations.

The first category, referred to herein as Static-Topology GNNs, treats the network graph as a fixed snapshot and focuses on leveraging structural dependencies at a single point in time. For example, Guo et al.²⁴ integrated GNNs with multi-armed bandits to learn and predict traffic patterns in SDN-IoT, effectively detecting anomalies under the assumption of a static graph. Zhou et al.²⁵ constructed a shadow GNN to generate adversarial attacks, relying on a fixed topology for gradient computation. Lo et al.²⁶ designed E-GraphSAGE, which models network traffic as graphs and exploits edge features together with a static topological view for intrusion detection. Lan et al.²⁷ extended this idea with E-min Batch GraphSAGE, shifting aggregation from node-level to edge-level information while maintaining the graph structure. Caville et al.¹⁴ proposed Anomal-E, a self-supervised method that learns representations from edge features and a static graph topology, alleviating the need for labeled data. Deng et al.²⁸ introduced EMAIDS, an edge-feature multi-hop attention GNN that jointly integrates node and edge features on a fixed graph. While these models effectively capture spatial correlations, their underlying assumption of a static graph structure overlooks the dynamic nature of modern networks, where connections between nodes evolve due to scaling, migration, or policy changes. Consequently, they fail to detect attacks that manifest through topological reconfigurations.

A second line of research, termed Node-Centric Temporal Models, attempts to incorporate temporal dynamics by modeling the historical behavior of individual nodes, often combining GNNs with recurrent units. For instance, Wang et al.²⁹ proposed a near-earth remote sensing network intrusion detection method based on a Spatio-Temporal Graph Attention Network (N-STGAT), which significantly improves intrusion detection accuracy and efficiency by integrating node states with the temporal characteristics of data flows. Duan et al.³⁰ introduced a novel network anomaly detection framework based on a Continuous Temporal Graph (CTG) neural network, with a primary focus on node-centric temporal interactions. Although these node-centric approaches excel at capturing micro-dynamics, they struggle to model graph-level long-term trends and the rewiring of network connections. While they can detect anomalous behavior at the individual node level, they are less capable of discerning whether such anomalies are part of a global attack pattern that manifests as systematic changes in communication patterns within the network.

The third category, Graph-Level Temporal Models, aims to capture the evolution of the entire network topology over time, typically by employing dynamic GNNs or line graph transformations. Duan et al.³¹ introduced a semi-supervised intrusion detection method based on Dynamic Line Graph Neural Networks (DLGNN), which first transforms network traffic into spatio-temporal graphs and then applies a dynamic GNN to capture the evolution of communication patterns, while line graphs facilitate message aggregation across time steps. Xu et al.³² proposed the Temporal Graph Attention (TGAT) layer for inductive representation learning on temporal graphs. This method efficiently aggregates temporal-topological neighborhood features and learns temporal feature interactions. These models excel at tracking macro-level structural changes but often operate at a coarse temporal granularity, failing to simultaneously capture the short-term bursts and fine-grained anomalies that occur at the node level between topological shifts.

While each category has advanced the field, these approaches remain fundamentally fragmented: static models ignore topology evolution, node-centric models miss global context, and graph-level models overlook micro-dynamics. To address these limitations, this paper introduces DMSTG-AD, a unified framework that synthesizes dynamic topology adaptation, multi-scale temporal modeling, and cross-attention feature fusion.

Specifically, DMSTG-AD employs dynamic node embeddings and an adaptive adjacency matrix to capture time-varying topological relationships; integrates multi-scale temporal convolutions to model both short-term bursts and long-term evolution; and applies STCA to align and fuse features within a unified representation space. By explicitly modeling the interplay between evolving topologies and multi-scale temporal patterns, DMSTG-AD establishes a new paradigm for joint dynamic spatio-temporal learning in NIDS, thereby enabling more fine-grained and adaptive anomaly detection in dynamic SDN environments.

Methodology

The development of DMSTG-AD is guided by three core design principles that address the fundamental challenges of intrusion detection in SDN environments. First, a dynamic graph perspective is adopted: network traffic is inherently non-stationary—both node semantics and topological connectivity evolve over time. Consequently, the model must transcend static graph assumptions and explicitly capture temporal variations at both the node and graph levels. Second, tightly coupled spatio-temporal modeling is prioritized rather than treating spatial and temporal features separately. Effective intrusion detection requires the ability to simultaneously exploit structural attack propagation paths and bursty traffic patterns; therefore, a unified representation space is designed in which spatial and temporal features can interact through cross-attention. Third, modularity and adaptability are emphasized: each functional component—dynamic node embedding, adaptive adjacency learning, edge-node collaborative convolution, multi-scale temporal encoding, and spatio-temporal fusion—is designed as an independent yet pluggable module. This design not only facilitates ablation analysis but also allows the framework to be easily extended or adapted to other network security tasks. Collectively, these principles ensure that DMSTG-AD is both theoretically sound and practically flexible for real-world SDN deployment.

This section presents the proposed DMSTG-AD model, a dynamic spatio-temporal graph neural network-based intrusion detection framework designed to jointly capture the spatial and temporal variations inherent in SDN traffic. The section begins by analyzing the SDN system architecture and constructing a threat model that identifies potential attack vectors and their manifestations in traffic data. Network traffic records are then chronologically sorted and partitioned into a series of discrete snapshot graphs, each representing the communication state within a fixed time window. The spatial feature extraction module—comprising an adaptive adjacency matrix and edge-node collaborative graph convolution—learns node representations that reflect both local neighborhood structures and global topological dependencies. Concurrently, the temporal feature extraction module employs a GRU-driven dynamic node embedding mechanism to capture short-term behavioral variations at the node level, while multi-scale dilated TCNs and a BiGRU jointly model graph-level short-term bursts and long-term trends. Finally, the STCA mechanism fuses the spatial and temporal features at the node level, producing robust representations that are fed into a classifier for fine-grained anomaly scoring and attack type identification. The detailed design of each module, along with their integration, is elaborated in the following subsections.

Formal problem formulation

Network intrusion traffic exhibits complex patterns that are inherently spatio-temporal and dynamic. To formally capture this property, the network monitored by an SDN controller is represented as a sequence of time-evolving graphs. Let $\{\mathcal{G}^{(t)}\}_{t=1}^T$ denote this dynamic graph sequence, where each snapshot at time t is defined as $\mathcal{G}^{(t)} = (V, E^{(t)}, A^{(t)})$. Here, V is the fixed set of nodes representing network entities (e.g., unique IP-port pairs). The set of edges $E^{(t)}$ captures communication links active at time t , reflecting the inherently dynamic nature of network interactions. Each node $v \in V$ is associated with a feature vector $a_v^{(t)} \in \mathbb{R}^F$, forming the node feature matrix $A^{(t)} \in \mathbb{R}^{|V| \times F}$, which encodes traffic attributes such as packet counts, byte volumes, and flow duration.

The core problem of Dynamic Multi-Scale Spatio-Temporal Anomaly Detection in SDN is to learn a discriminative function f_Θ that maps the observed graph sequence to node-level labels. Formally, the objective is:

$$f_\Theta : \{\mathcal{G}^{(1)}, \dots, \mathcal{G}^{(T)}\} \rightarrow \mathbf{Y} \in \{0, 1\}^{|V|} \quad (\text{binary}) \quad \text{or} \quad \mathbf{Y} \in [0, 1]^{|V| \times C} \quad (\text{multi-class}) \quad (1)$$

where $\mathbf{Y}$ denotes the anomaly labels, with C being the number of attack classes plus the normal class. This formulation fundamentally differs from and addresses the limitations of two prevalent simplified approaches:

1. *Static Graph Models*: These methods assume a fixed graph structure $\mathcal{G} = (V, E, A)$, ignoring the temporal evolution of edges $E^{(t)}$ and node features $A^{(t)}$. Consequently, they fail to capture the propagation and shift of attack behaviors over time.
2. *Decoupled Spatio-Temporal Models*: Many existing approaches process spatial and temporal information in separate stages or branches, subsequently combining the outputs via late fusion. This decoupling implicitly assumes that spatial and temporal features are independent, thereby neglecting the deep, time-varying interconnections between them—connections that are crucial for identifying sophisticated attacks that manipulate both network topology and traffic timing.

This work addresses the more complex and realistic scenario in which spatial dependencies and temporal dynamics co-evolve. Addressing this complexity necessitates a model capable of jointly learning a dynamic graph structure to reflect evolving node relationships, capturing multi-scale temporal patterns ranging from transient bursts to prolonged, stealthy attack activities, and performing deep fusion of spatial and temporal representations to uncover anomalous spatio-temporal correlations. The challenges are threefold: (1) modeling

the intrinsic dynamism of attack-induced graph topology, (2) characterizing temporal patterns across multiple granularities, and (3) achieving effective alignment and fusion between spatial and temporal manifolds for optimal feature discrimination.

Threat mode

SDN is a novel network architecture composed of three planes: the data plane, the control plane, and the application plane. Communication between these planes is facilitated through northbound and southbound interfaces³³. Unlike traditional network architectures, SDN centralizes its key functions within the controller, which is responsible for determining data forwarding and routing strategies. While this design significantly simplifies network management, it also renders the controller inherently more susceptible to cyberattacks. Once compromised, the controller's ability to manage data forwarding and routing becomes impaired, underscoring the critical need for an effective intrusion detection system tailored specifically to SDN environments.

Based on the centralized control characteristics of SDN, the threat model illustrated in Fig. 1 is designed. Through the southbound interface (e.g., OpenFlow), the SDN controller collects flow statistics and specific packets (Packet-In messages) from all switches. These records inherently contain five-tuple features (source IP, source port, destination IP, destination port, and transport protocol), as well as statistical features such as flow duration, packet counts, and byte counts. Timestamps are typically recorded by the controller upon receiving such information. A security monitoring module is deployed at the SDN controller, responsible for aggregating, caching, and preprocessing traffic information. This module is tightly integrated with the NIDS detection engine. The detection engine performs real-time threat detection using processed traffic features, and based on the detection results, the security application invokes the controller's capabilities to install flow table rules on switches. This enables real-time detection, caching (for further analysis), and blocking of malicious traffic.

Attackers may launch a variety of threats, including DDoS, reconnaissance, injection, brute-force, and botnet attacks. These malicious flows are often intermingled with a large volume of legitimate traffic. To distinguish between normal and anomalous flows, the security monitoring module extracts relevant traffic statistics. The NIDS model is trained offline using these features; once trained, the model serves as a traffic filter. It makes classification decisions prior to data forwarding and routing at the SDN controller, thereby mitigating the adverse effects of attacks on the network.

Dynamic spatio-temporal graph sequence

The network traffic collected by the SDN controller contains timestamp fields. By sorting cached traffic data in chronological order and storing it within the security monitoring module, the resulting traffic sequence naturally possesses temporal characteristics. In this study, graphs are constructed at fixed intervals, thereby forming a sequence of static spatio-temporal graphs. Each static graph can be represented as $G = \{V, E, A\}$, where V denotes the set of nodes, E denotes the set of edges between nodes, and A is the adjacency matrix that stores topological information. For example, $A(u, v) = 1$ indicates that there exists an edge between node u and node

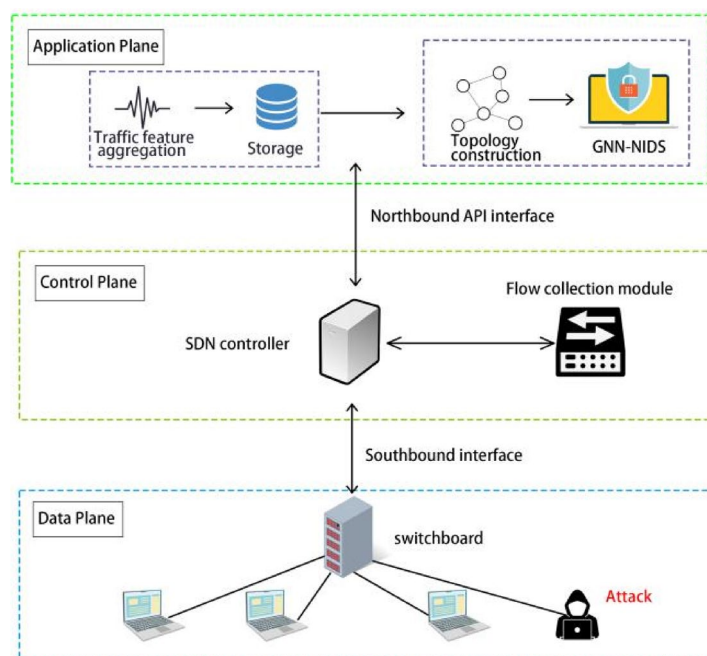


Fig. 1. Threat model of SDN-based intrusion detection, where attacks originate from the data plane and are monitored through the control plane, with traffic features aggregated and analyzed in the application plane by GNN-NIDS for real-time detection.

v , whereas $A(u, v) = 0$ indicates no connection. A series of consecutive static graphs can then be combined into a dynamically evolving graph sequence:

$$\mathcal{G} = \{G_1, G_2, \dots, G_T\} \tag{2}$$

which captures the temporal variations of network traffic across different time intervals. When a session request is initiated from the data plane, the traffic collected by the monitoring module contains information such as source IP address, source port, destination IP address, and destination port. Following common practices in existing research, the IP address and port are concatenated to generate the final node identifier, while communication between two addresses is represented as an edge between the corresponding nodes. Traffic attributes such as flow duration, packet size, and transport protocol are aggregated into node features, whereas the number of communications between two IP addresses is used to construct edge weights. This representation effectively characterizes various types of network attacks. For example, a DDoS attack involves multiple machines or devices simultaneously sending large volumes of requests to a target server, potentially causing delays or crashes of the SDN controller. Such attacks result in a sudden surge in traffic volume and communication frequency between IP addresses, which manifests in the constructed graph as a sharp increase in both the number of nodes and the edge weights within a single snapshot. These anomalous patterns can be effectively captured by graph neural network models.

In this study, the process of constructing dynamic spatio-temporal graph sequences is performed as follows. The traffic collected by the SDN controller is first sorted in chronological order according to timestamps to preserve its temporal characteristics. Next, IP addresses and port numbers are concatenated to generate precise node identifiers, and the communication frequency between addresses is computed and stored. Based on these rules, each address is treated as a node, with aggregated flow attributes such as duration, packet size, and protocol type serving as node features. Communications between addresses are represented as edges, with their respective frequencies used as edge weights. According to a predefined time window, these elements are organized into static graph snapshots, which serve as inputs to the model for extracting spatial features and local temporal dependencies. Finally, multiple static snapshots across consecutive time windows are combined to form dynamic spatio-temporal graph sequences, enabling the model to capture the global temporal evolution patterns of network traffic. The overall construction process of dynamic graph sequences is illustrated in Fig. 2.

Spatial feature extraction module

The spatial feature extraction module is designed to jointly model node-level static attributes, dynamic node embeddings, and edge-level attributes. By employing an adaptive adjacency matrix in conjunction with an edge–node collaborative convolution mechanism, the module captures both local and global spatial dependencies. In this process, node representations are first obtained by fusing static features with dynamic embeddings. An adaptive adjacency matrix is then generated based on feature similarity, which dynamically reflects the evolving relationships between nodes. Finally, edge–node collaborative graph convolution aggregates neighborhood information while simultaneously incorporating edge semantics, thereby enabling high-precision modeling of spatial features. This design not only preserves the advantages of traditional graph convolution in neighborhood aggregation but also enhances the model’s capacity to represent complex traffic patterns through learnable topology construction and explicit edge attribute modeling.

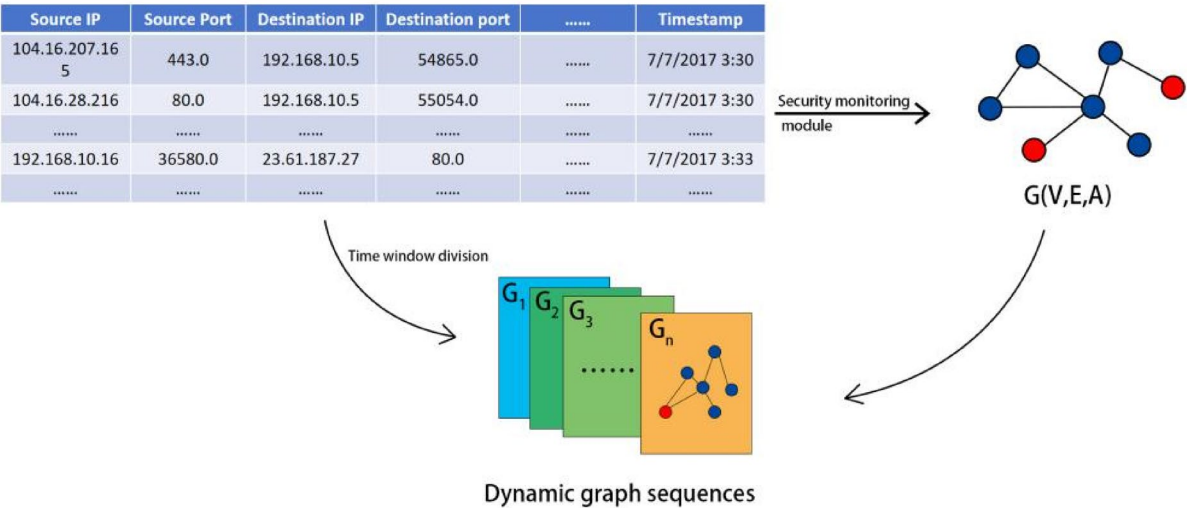


Fig. 2. Network traffic records are divided into time windows and transformed into dynamic graph sequences, where each snapshot captures evolving interactions for intrusion detection analysis.

Feature fusion

At the input stage, each node is associated with an original static feature vector $X^{static} \in \mathbb{R}^{N \times F}$ and a temporal contextual representation $X^{dyn} \in \mathbb{R}^{N \times F}$ generated by the dynamic node embedding module. Since these two feature types reside in different semantic spaces, directly concatenating or summing them may lead to inconsistencies. To address this, a linear projection is first applied to the dynamic embeddings:

$$X^{proj} = W_{proj} X^{dyn} + b_{proj} \quad (3)$$

The projected representation is then fused with static features through residual addition:

$$X^{fused} = X^{static} + \text{ReLU}(\text{BN}(X^{proj})) \quad (4)$$

This fusion strategy offers two key advantages. First, the projection maps the dynamic embeddings into the same feature space as the static attributes, enabling a unified representation of temporal information and inherent node properties. Second, the residual connection ensures that the original static features are preserved, thereby improving training stability and mitigating the vanishing gradient problem in deep networks. Consequently, the fused features retain both node semantics and temporal signals, providing more discriminative inputs for subsequent spatial modeling.

Adaptive adjacency matrix

Conventional graph convolutional networks operate on a predefined, static adjacency matrix, which implicitly assumes that the underlying graph topology resides on a fixed manifold. From a graph signal processing perspective, such a static adjacency matrix acts as a low-pass filter that smooths node features over the manifold, suppressing high-frequency variations. While this inductive bias is effective for tasks where the relational structure remains stationary, it becomes restrictive in dynamic intrusion detection scenarios. Attack behaviors often induce non-stationary, task-specific topological shifts—for example, a sudden surge of connections toward a victim host during a DDoS attack—that are not reflected in a precomputed graph.

In network traffic modeling tasks, node dependencies are inherently dynamic, and a fixed topology cannot adequately capture this temporal variability. To overcome this limitation, an adaptive adjacency matrix generation mechanism is introduced³⁴, which dynamically adjusts topological relationships based on evolving node states. By computing similarity in a learnable latent space and normalizing via softmax, the model dynamically rewires the graph to amplify discriminative patterns associated with anomalies. This capacity to jointly learn the graph structure alongside node representations is a hallmark of expressive dynamic graph networks, enabling the model to capture time-varying dependencies that are critical for accurate detection.

Specifically, the fused features are projected into a latent space through linear transformation and nonlinear activation:

$$H^{latent} = \text{ReLU}(W_h X^{fused} + b_h) \quad (5)$$

In this latent space, node similarity is computed via inner product:

$$S_{i,j} = H^{latent} (H^{latent})^T \quad (6)$$

where $S_{i,j}$ denotes the semantic similarity between nodes i and j . To allow this similarity matrix to be used as convolution weights, we apply row-wise softmax normalization:

$$A = \text{Softmax}(S, \text{dim} = 1) \quad (7)$$

The resulting adaptive adjacency matrix A is continuously optimized during training. Its elements dynamically adjust in response to the evolution of node features, enabling the model to automatically identify and emphasize highly correlated node relationships. This learnable topological structure overcomes the limitations of fixed adjacency matrices and provides more flexible, semantically aligned neighborhood relations, thereby enhancing the representational capacity of spatial features.

Edge-node collaborative graph convolution

After obtaining the fused node representations and the adaptive adjacency matrix, an edge-node collaborative graph convolution mechanism is further designed to fully exploit edge attributes in spatial modeling. Unlike conventional convolution operations that rely solely on node features, this mechanism explicitly integrates edge semantics into node updates through dynamic edge updating and gated modulation, thereby strengthening the model's capacity to capture complex interaction relationships.

Under this mechanism, node features are updated through multi-layer propagation guided by the adaptive adjacency matrix:

$$H^{gcn,(l+1)} = \text{ReLU}(A H^{gcn,(l)} W^{(l)}), \quad l = 0, 1, \dots, K \quad (8)$$

where A is the adaptive adjacency matrix and $W^{(l)}$ is the parameter matrix of the l -th convolution layer. This produces intermediate representations that preserve local topology. Simultaneously, edge features are explicitly modeled. For each edge $(i \rightarrow j)$, the representations of the source and target nodes are concatenated, and selective gating weights are computed as:

$$gate_{ij} = \sigma \left(W_g \left[H_i^{gcn} \parallel H_j^{gcn} \right] + b_g \right) \quad (9)$$

The original edge feature E_{ij}^{attr} is linearly mapped and, together with the source node representation, updated by a GRU unit³⁵:

$$E'_{ij} = GRUCell(W_{edge} E_{ij}^{attr}, H_i^{gcn}) \quad (10)$$

The updated edge feature is projected into the same dimension as the node features and combined with the gating weights to produce the final edge message:

$$m_{ij} = gate_{ij} \odot (W_p E'_{ij}) \quad (11)$$

All edge messages are aggregated at the target node:

$$M_j = \sum_{i \in \mathcal{N}(j)} m_{ij} \quad (12)$$

Finally, the spatial representation is obtained by combining the GCN outputs, the aggregated edge messages, and a residual projection of the original features:

$$X^{spatial} = H^{gcn} + M + \text{ResidualProj}(X^{static}) \quad (13)$$

This edge-node collaborative mechanism not only preserves the efficient neighbor aggregation of traditional graph convolution but also dynamically evolves edge features through GRU units across both temporal and semantic levels. As a result, node updates jointly reflect neighbor and edge influences, yielding more accurate and robust spatial feature representations in complex graph structures.

Input: Static node features $X^{static} \in \mathbb{R}^{N \times F}$, dynamic embeddings $X^{dyn} \in \mathbb{R}^{N \times F}$; Edge index $edge^{index}$, edge attributes $E^{attr} \in \mathbb{R}^{E \times De}$

Set Hyperparameters: number of GCN layers K

Output: Spatial node features $X^{spatial} \in \mathbb{R}^{N \times F_{out}}$

```

1:  $X^{fused} \leftarrow X^{static} + \text{ReLU}(\text{BatchNorm}(W_{proj} X^{dyn} + b_{proj})) \in \mathbb{R}^{N \times F}$ 
2:  $H^{latent} \leftarrow \text{ReLU}(W_h X^{fused} + b_h) \in \mathbb{R}^{N \times H}$ 
3:  $S \leftarrow H^{latent} (H^{latent})^T \in \mathbb{R}^{N \times N}$ 
4:  $A \leftarrow \text{Softmax}(S, \text{dim}=1)$ ; optionally sparsify  $A$ 
5:  $H^{gcn} \leftarrow X^{fused}$ 
6: for  $l = 1..K$  do
7:    $H^{gcn} \leftarrow \text{ReLU}(\text{GCN}^l(H^{gcn}, edge^{index}, A))$ 
8: end for
9:  $M \leftarrow \text{aggregate over edges: } gate([H^{gcn\_src}, H^{gcn\_dst}]) * \text{Linear}(GRUCell(\text{Linear\_edge}(E^{attr}), H^{gcn\_src})) \in \mathbb{R}^{N \times F_{out}}$ 
10:  $X^{spatial} \leftarrow H^{gcn} + M + \text{ResidualProj}(X^{static}) \in \mathbb{R}^{N \times F_{out}}$ 
11: return  $X^{spatial}$ 

```

Algorithm 1. The Spatial Feature Extraction Process

Temporal feature extraction module

This section focuses on the temporal feature extraction component of the proposed model, which is specifically designed to capture the dynamic variations of network traffic in anomaly detection scenarios. The module aims to enhance sensitivity and robustness in modeling both short-term fluctuations and long-term trends of traffic streams. In network traffic graphs, node representations correspond to entities such as hosts or connections, while edges denote communication interactions. The temporal evolution of traffic not only reflects variations in node-level attributes but also reveals fluctuations at the system-wide structural level. Building upon the spatial feature extraction described earlier, two key modules are introduced: the dynamic node embedding module and the multi-scale temporal modeling module. Together, they enable fine-grained characterization of both local node behaviors and global temporal patterns within the graph structure.

Dynamic node embedding module

In anomaly detection tasks, the behavior of a single node is not only determined by its current state but is also influenced by the short-term evolution of its historical states. To capture this, a dynamic node embedding module is designed, employing a GRU as its core mechanism to model the temporal dynamics of each node. Given the

input node features at time t , denoted as $\mathbf{X}^t \in \mathbb{R}^{N \times F}$, and the hidden state from the previous timestamp H^{t-1} , the update of the GRU is defined as:

$$H^t = \text{GRU}(X^t, H^{t-1}) \quad (14)$$

where $H^t \in \mathbb{R}^{N \times H}$ represents the updated hidden state, and the dimension H controls the capacity of the model to encode historical temporal information. To ensure consistency with the spatial feature dimension, the GRU outputs are further projected linearly and fused with the static node features via residual addition:

$$\tilde{X}^t = X^t + \text{Linear}(H^t) \quad (15)$$

In this manner, each node representation not only retains its static semantics but also incorporates short-term historical evolution, thereby enabling the model to capture abrupt traffic bursts at the node level. In practice, this mechanism ensures temporal continuity by updating node states across timestamps, thus maintaining both local and global temporal consistency.

Multi-scale temporal modeling module

Building upon the dynamic node embeddings, a multi-scale temporal modeling module is further introduced to capture both short-term dynamics and long-term dependencies of traffic sequences. This module integrates multiple dilated TCNs³⁶ with a Bi-GRU. Each TCN models temporal dependencies at different dilation rates, while the Bi-GRU captures sequential information in both forward and backward directions. The resulting representations are subsequently fused to produce robust graph-level temporal features.

Specifically, for each timestamp, the updated node features $\tilde{X}^t$ are aggregated using average pooling to obtain the graph-level representation:

$$g^t = \text{GlobalMeanPool}(\tilde{X}^t) \quad (16)$$

The recent L graph representations are stored in a sliding window $\delta = \{g^{t-L+1}, \dots, g^t\}$, which is then concatenated to form the temporal sequence $S \in \mathbb{R}^{L \times F}$. To capture both short-term and long-term dependencies, m parallel dilated convolutions with different dilation rates $d \in D$ are applied:

$$T_d = \text{DilatedTCN}_d(S), \quad d \in D \quad (17)$$

These convolutions capture local fluctuations and periodic variations. To complement the TCN-based modeling, the temporal sequence is further processed using a Bi-GRU:

$$T_{gru} = \text{BiGRU}(S^T)_{\text{mean_time}} \quad (18)$$

where the hidden states across time are averaged to produce long-term temporal features. Finally, the outputs of all branches are concatenated and projected linearly:

$$G^t = \text{Linear}(\text{concat}(T_{d_1}, \dots, T_{d_m}, T_{gru})) \quad (19)$$

When the sliding window is not fully filled, the pooled representation g^t is directly used as the temporal feature; otherwise, the fused multi-scale representation G^t serves as the final graph-level temporal feature. For clarity, the overall procedure of temporal feature extraction, including dynamic node embedding and multi-scale sequence modeling, is summarized in Algorithm 2.

Input: Spatial node feature snapshots $X^1, X^2, \dots, X^T$ with $X^t \in \mathbb{R}^{N \times f}$, Node-ID set V and previous hidden-state table H^0

Set hyperparameters: GRU hidden size H , dilation set $D = \{d_1, \dots, d_m\}$, sliding-window length L ($L \leq T$)

Output: Temporal node embeddings $\tilde{X}^1, \tilde{X}^2, \dots, \tilde{X}^T$ ($\mathbb{R}^{N \times f}$) and graph-level temporal features $G^1, G^2, \dots, G^T$ ($\mathbb{R}^f$)

- 1: Initialise hidden-state table $H \leftarrow H^0$
- 2: **for** each time step $t = 1 : T$ **do**
- 3: $\tilde{X}^t, H \leftarrow \text{GRU}(X^t, H)$
- 4: $\tilde{X}^t \leftarrow X^t + \text{Linear}(\tilde{X}^t)$
- 5: $g^t \leftarrow \text{GlobalMeanPool}(\tilde{X}^t)$
- 6: Enqueue g^t into sliding window δ ; keep last L vectors
- 7: **if** $|\delta| = L$ **then**
- 8: $S \leftarrow \text{stack}(\delta)$
- 9: **for** each d in D parallel **do**
- 10: $T_d \leftarrow \text{DilatedTCN}^d(S)$
- 11: $T_{\text{gru}} \leftarrow \text{BiGRU}(S^T). \text{mean_time}()$
- 12: $G^t \leftarrow \text{Linear}(\text{concat}(T_{d_1}, \dots, T_{d_m}, T_{\text{gru}})) \triangleright \mathbb{R}^f$
- 13: **end for**
- 14: **else**
- 15: $G^t \leftarrow g^t$
- 16: **end if**
- 17: **end for**
- 18: **return** $\{\tilde{X}^1, \dots, \tilde{X}^T\}, \{G^1, \dots, G^T\}$

Algorithm 2. The Temporal Feature Extraction Process

Spatio-temporal feature fusion

A naive fusion of spatial and temporal features—such as simple concatenation or element-wise addition—operates under the implicit assumption that the two modalities are independent and already aligned in representation space. This assumption rarely holds in dynamic intrusion detection scenarios, where structural roles and temporal contexts are deeply intertwined. From a manifold learning perspective, spatial and temporal features reside on distinct yet coupled manifolds; effective fusion requires manifold alignment that preserves modality-specific discriminative information while minimizing redundancy.

To address this challenge, a STCA mechanism is designed to model cross-modal interactions within a joint embedding space. Using spatial features as queries and temporal features as keys and values, STCA computes attention weights that reflect the contextual relevance of temporal patterns to each node's structural state. This formulation enables non-linear, content-aware fusion that adaptively emphasizes temporal signals only when they are structurally salient. From an information-theoretic standpoint, this cross-attention scheme can be interpreted as learning a soft alignment that minimizes conditional redundancy between spatial and temporal representations, yielding a fused feature space with higher discriminative capacity than that achievable from independent modalities.

After completing the extraction of spatial and temporal features, the model integrates the two such that node representations not only capture the spatial dependencies revealed by structural information but also encode the dynamic temporal variations reflected by sequential evolution. To achieve this, the STCA mechanism enables interactions between spatial and temporal features at the node level, thereby producing robust spatio-temporal representations.

Specifically, let the spatial feature of node i be denoted as $X_i^{\text{spatial}} \in \mathbb{R}^F$, and the temporal graph-level feature as $G^t \in \mathbb{R}^F$. To enable cross-modal interactions, these features are first mapped into a shared latent space and subsequently transformed into three distinct semantic vectors: Query (Q), Key (K), and Value (V):

$$Q_i = W_Q X_i^{\text{spatial}}, \quad K_i = W_K G^t, \quad V_i = W_V G^t \quad (20)$$

where W_Q, W_K, W_V are learnable parameter matrices. Subsequently, the similarity between the spatial representation of node i and the temporal representation is computed as:

$$\alpha_i = \frac{\exp(Q_i K_i^T)}{\sum_j \exp(Q_j K_j^T)} \quad (21)$$

The attention coefficient α_i reflects the alignment between node-level spatial features and temporal representations. Using this weight, the temporal feature vector is aggregated to produce the supplementary temporal component:

$$Z_i = \alpha_i V_i \quad (22)$$

Finally, the fused spatio-temporal representation of node i is obtained through residual combination of its spatial and temporal components:

$$X_i^{st} = X_i^{spatial} + \gamma Z_i \quad (23)$$

where γ is a learnable scaling factor that balances the contribution of the temporal component relative to the spatial representation.

This design enables node embeddings to adaptively integrate global temporal information into their local spatial context, thereby enriching the representations with both structural semantics and temporal dynamics. Importantly, the STCA mechanism retains the structural advantages of convolution-based spatial encoding while introducing temporal dependency modeling through attention. As a result, the fused representation is capable of simultaneously capturing both short-term variations and long-term trends in network traffic—a capability that is critical for effective anomaly detection. The resulting spatio-temporal features are then passed to a multi-layer perceptron (MLP) classifier to perform the final anomaly prediction.

Overall model framework

The overall architecture of the proposed method adopts an end-to-end spatio-temporal fusion framework, which takes raw network traffic data as input and sequentially performs dynamic node embedding, spatial feature extraction, global temporal encoding, and spatio-temporal feature fusion. The ultimate objective is to generate discriminative node-level representations for effective anomaly detection.

Concretely, network traffic data collected from SDN controllers are first represented as graphs, where nodes correspond to traffic entities (e.g., hosts or connections) and edges represent communication flows. The dynamic node embedding module employs a GRU to capture the short-term behavioral evolution of each node by updating hidden states across consecutive timestamps. These embeddings are then combined with static node attributes to preserve both inherent semantics and temporal signals.

Building upon these representations, the spatial feature extraction module leverages adaptive adjacency learning and edge–node collaborative convolution to capture both local neighborhood dependencies and global structural relationships. Concurrently, the temporal feature extraction module introduces multi-scale dilated convolutions and Bi-GRUs to encode both bursty fluctuations and long-term trends within traffic streams. The STCA module subsequently integrates the spatial and temporal representations at the node level, enabling balanced and adaptive fusion.

Finally, the classifier leverages these fused spatio-temporal features to perform anomaly detection. This hierarchical design, characterized by strong complementarity across modules, ensures that the model not only detects short-term traffic anomalies but also captures the structural-temporal dependencies indicative of persistent malicious behaviors.

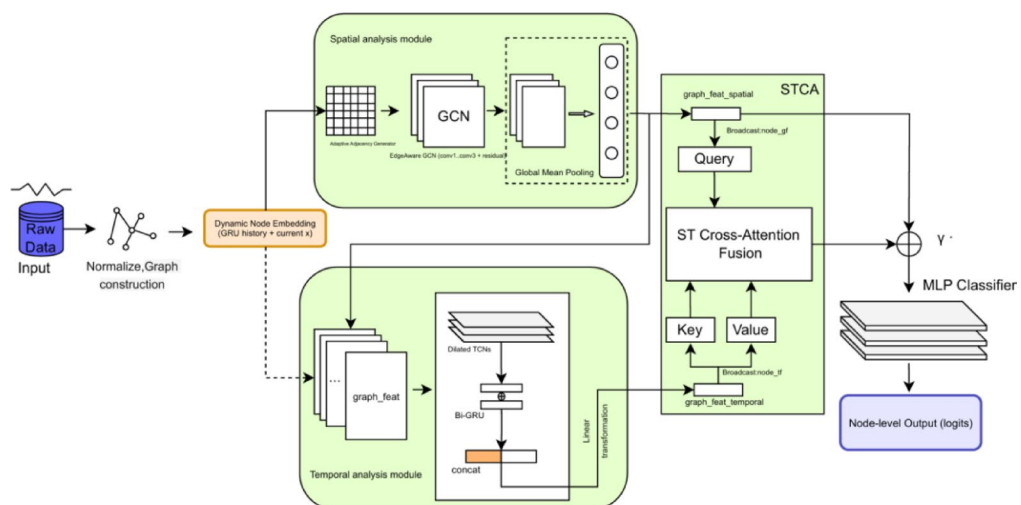


Fig. 3. The proposed intrusion detection framework first normalizes raw traffic data and constructs graphs, then generates dynamic node embeddings combining historical and current states. A spatial analysis module with GCN captures topological features, while a temporal analysis module with dilated TCN and Bi-GRU learns sequential dependencies. Finally, spatial and temporal features are fused through STCA and fed into an MLP classifier for node-level intrusion detection.

To better illustrate the integration of all modules, the complete workflow of the proposed framework is detailed in Algorithm 3 and visualized in Fig. 3. Algorithm 3 presents the step-by-step computational procedure, while Fig. 3 highlights the interactions among the key components: the spatial analysis module (GCN-based), the temporal analysis module (GRU and TCN-based), and the STCA fusion module that integrates both streams for final anomaly classification.

Input: Sequence of snapshots $\{X^1, \dots, X^T\}$, $X^k \in \mathbb{R}^{n \times f}$; Corresponding line-graph adjacencies $\{A^1, \dots, A^T\}$, $A^k \in \mathbb{R}^{n \times n}$
Set hyperparameters: DropEdge rate θ_e , batch-size B , sliding-window length L , labeled ratio p
Output: Node-level anomaly predictions $\hat{y}$

- 1: Partition $\{1, \dots, T\}$ into batches of size B , initialize sliding window $W \leftarrow \emptyset$
- 2: **while** not converged **do**
- 3: **for** each batch of snapshot indices $\mathcal{B}$ **do**
- 4: **for** $k \in \mathcal{B}$ **do**
- 5: $A^k \leftarrow \text{DropEdge}(A^k, \theta_e)$
- 6: $X^{\text{spatial } k} \leftarrow \text{Algorithm 1: SpatialFeatureExtract}(X^k, A^k)$
- 7: $H^k \leftarrow \text{Algorithm 2: TemporalFeatureExtract}(X^{\text{spatial } k}, H^{k-1}, W, L)$
- 8: enqueue $W \leftarrow W \cup \{\text{global_mean}(X^{\text{spatial } k})\}$; drop oldest if $|W| > L$
- 9: **end for**
- 10: $\hat{y}_{\mathcal{B}} \leftarrow \text{Classifier}(\text{STCA}(\{X^{\text{spatial } k}\}, \{H^k\}))$
- 11: $\mathcal{L} \leftarrow \text{SemiSupLoss}(\hat{y}_{\mathcal{B}}; \text{labels} \subseteq \mathcal{B}, \text{ratio}=p)$
- 12: $\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}$
- 13: **end for**
- 14: **end while**
- 15: **return** $\hat{y}$

Algorithm 3. The Overall Process of the Proposed Method

Experiments

Datasets

To evaluate the generalization capability of the proposed model across diverse traffic environments and attack scenarios, two widely used public datasets in the SDN intrusion detection domain were selected: CIC-IDS2017 and InSDN.

CIC-IDS2017

The CIC-IDS2017 dataset³⁷ was jointly released by Carleton University and the Canadian Institute for Cybersecurity as a high-quality benchmark for intrusion detection research. It comprises both normal and malicious traffic collected in a real-world network environment. The dataset was generated from realistic Internet sessions and encompasses multiple types of attacks, including DoS, DDoS, brute force, web attacks, botnets, and port scans. Over ten million session records were captured over a five-day period.

To simplify the classification task while accounting for similarities among certain attacks, the original attack categories were merged into broader groups. Specifically, various fast and slow DoS attacks were grouped under the label “DoS/DDoS”; SSH and FTP brute force attempts were combined into “Brute Force”; multiple types of scanning activities were consolidated into “PortScan”; while isolated categories such as Botnet and Web Attacks were retained as individual groups. The correspondence between the original and merged attack types is summarized in Table 1.

InSDN

The InSDN dataset³⁸ was specifically designed for intrusion detection research in SDN environments. It comprises diverse normal and malicious traffic captured between SDN controllers and switches. The dataset encompasses multiple representative attack scenarios, including HTTP Flood, TCP SYN Flood, TCP FIN Flood, Port Scan, ARP Spoofing, DNS Spoofing, and LLDP Flood, as well as normal traffic interactions such as ARP, LLDP, and DNS.

In total, the dataset contains millions of sequential traffic samples, each characterized by multi-dimensional features such as packet length, latency, flow duration, and TCP flag distribution. Detailed statistics for the main attack categories in the dataset are provided in Table 2.

These two datasets offer complementary characteristics: CIC-IDS2017 encompasses a wide range of attack behaviors in large-scale, Internet-like environments, while InSDN focuses on traffic dynamics within SDN-specific network infrastructures. Together, they provide a comprehensive evaluation basis for validating the robustness and effectiveness of the proposed intrusion detection framework.

Original attack type	Amount	Attack type after merge	Amount
BENIGN	2,273,097	Benign	2,273,097
Bot	1966	Bot	1966
DDoS	128,027	DDoS	128,027
DoS GoldenEye	10,293	DoS	252,672
DoS Hulk	231,073		
DoS Slowhttptest	5499		
DoS slowloris	5796		
Heartbleed	11		
Infiltration	36	PortScan	158,966
PortScan	158,930		
FTP-Patator	7938	Patator	13,835
SSH-Patator	5897		
Web Attack – Brute Force	1507	Web Attack	2180
Web Attack – SQL Injection	21		
Web Attack – XSS	652		

Table 1. Attack types in CIC-IDS2017 before and after merging.

Attack type	BFA	Botnet	DDoS	DoS	Normal	Probe	U2R	Web-Attack
Amount	1405	164	121,942	53,616	68,424	98,129	17	192

Table 2. Attack types in InSDN dataset.

Dataset	Data distribution			No.original samples	No.of features
CIC-IDS2017	Class	Train	Test	Original: 2,830,743 Sampled: 299,258	Original: 85 Sampled: 51
	Benign	181,900	45,410		
	Bot	1583	383		
	DDoS	10,129	2674		
	DoS	20,180	5087		
	Patator	11,191	2644		
	PortScan	12,682	3215		
	Web Attack	1741	439		
InSDN	Class	Train	Test	Original: 343,889 Sampled: 343,889	Original: 84 Sampled: 72
	Normal	54,665	13,759		
	BFA	1124	281		
	Botnet	137	27		
	DDoS	97,537	24,405		
	DoS	42,972	10,644		
	Probe	78,508	19,621		
	U2R	14	3		
	Web-Attack	154	38		

Table 3. Attack type distribution and feature counts after sampling.

Data preparation

Due to the large scale of the raw data and limited experimental hardware resources, stratified random sampling based on attack types was employed to preserve the overall distribution of attack categories while mitigating the risk that rare attack samples would be too scarce for effective learning. To eliminate redundant and irrelevant features, Random Forest was utilized for feature selection on both datasets. Features were ranked according to their importance, and a subset of the most informative features was selected for this study. The distribution of traffic samples for each attack type after sampling is presented in Table 3.

After sampling, 80% of the data was used for training, while the remaining 20% was reserved for testing. Following the previously defined graph construction rules, IP addresses and ports were concatenated to form complete node identifiers. Source and destination addresses correspond to two distinct nodes, and communication records between them are represented as edges in the graph. Continuous traffic events were

Hyperparameter	Description	Default value
Learning rate	Learning rate of the Adam optimizer	0.0001
Weight decay	Weight decay coefficient (L2 regularization) for Adam optimizer	0.0001
Batch size	Number of graph samples per batch	16
Hidden dimension	Hidden dimension size for GCN and other intermediate layers	128
Time steps	Window length of the multi-scale temporal module	5
Num epochs	Total number of training epochs	300
Window size	Time window length during data processing (minutes)	5

Table 4. Default hyperparameters.

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

Table 5. Confusion matrix.

partitioned into graph sequences using a sliding time window. The attack type for each node is determined by the communication labels of all its incoming and outgoing edges: if a node experiences multiple attack types within the same time window, the final attack category is assigned by majority voting.

Through this sampling and feature selection process, the CIC-IDS2017 and InSDN datasets were preprocessed to ensure data balance, thereby facilitating effective training and evaluation of the proposed model across diverse attack types.

Experimental setup

The experimental setup is as follows: experiments were conducted on a system equipped with an Intel Core i5-13400F CPU, 32 GB of RAM, an NVIDIA GeForce RTX 3060 GPU with 12 GB of VRAM, running the Windows 10 operating system. The software environment included Python 3.10, PyTorch 2.3.1, and PyTorch Geometric 2.6.1. The default hyperparameters for the proposed model are listed in Table 4. Notably, the time window length used during data processing and the window length of the multi-scale temporal module are both set to 5 (with the former measured in minutes)—a choice justified by balancing data representativeness against hardware constraints. A window size smaller than 5 would yield insufficient node samples per time window, hindering the capture of meaningful spatio-temporal correlations; conversely, a larger window would substantially increase the batch data volume, exceeding the memory capacity of the experimental hardware and leading to out-of-memory issues. This configuration ensures stable model training while preserving the validity of temporal feature extraction.

Evaluation metrics

To evaluate the performance of the proposed NIDS model, we adopt four classic metrics: accuracy, precision, recall, and F1-score. These metrics are widely used for performance assessment in NIDS, and their definitions and calculation processes are as follows:

As shown in the Table 5, a confusion matrix is used for binary classification, where: True Positive (TP): The model correctly predicts positive samples as positive; True Negative (TN): The model correctly predicts negative samples as negative; False Positive (FP): The model incorrectly predicts negative samples as positive; False Negative (FN): The model incorrectly predicts positive samples as negative.

The performance metrics are calculated as follows:

Accuracy: Accuracy represents the proportion of correctly classified samples out of all samples. It is calculated as:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (24)$$

Precision: Precision represents the proportion of true positive samples out of all the samples predicted as positive. It is calculated as:

$$Precision = \frac{TP}{TP + FP} \quad (25)$$

Recall: Recall indicates the proportion of actual positive samples that were correctly predicted as positive by the model. It is calculated as:

$$Recall = \frac{TP}{TP + FN} \quad (26)$$

Method	CIC-IDS2017				InSDN			
	Accuracy	Precision	Recall	F1-score	Accuracy	Precision	Recall	F1-score
E-GraphSage	0.9541	0.9542	0.9541	0.9522	0.9981	0.9981	0.9981	0.9981
GCN	0.9612	0.9606	0.9612	0.9607	0.9983	0.9983	0.9983	0.9983
GAT	0.9441	0.9429	0.9441	0.9423	0.9978	0.9978	0.9978	0.9978
CNN-BILSTM	–	–	–	–	0.9777	0.9985	0.9528	0.9751
XMF-GNN	0.9872	0.9846	0.9883	0.9878	–	–	–	–
DMSTG-AD	0.9912	0.9911	0.9912	0.9911	0.9992	0.9992	0.9992	0.9992

Table 6. Comparison of binary classification experiment results.

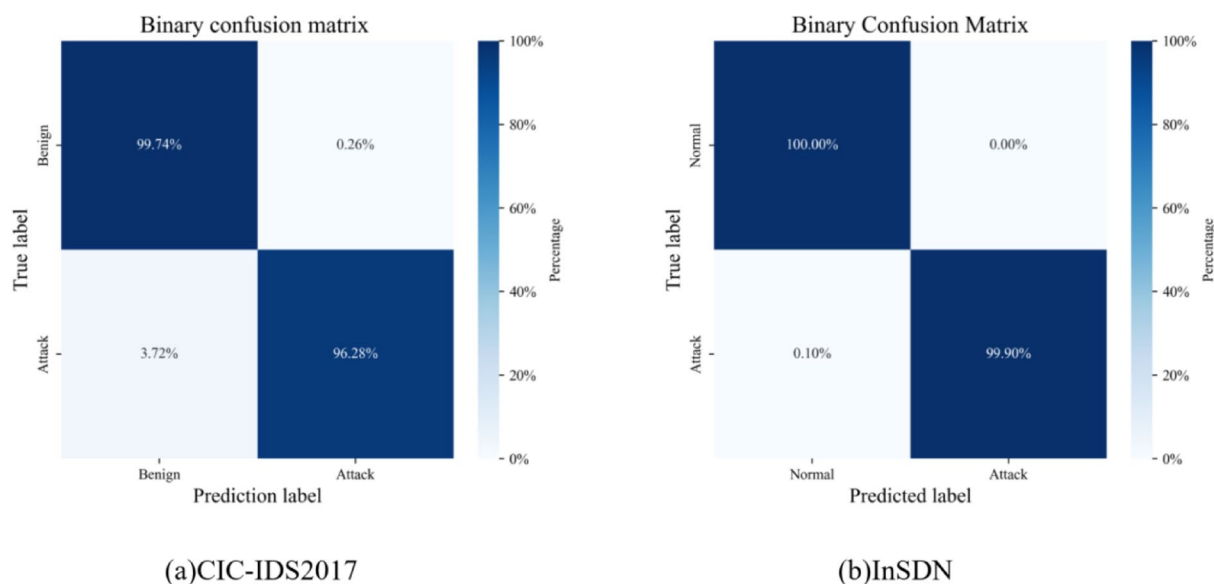


Fig. 4. Binary classification confusion matrix.

F1-score: The F1-score is the harmonic mean of precision and recall, combining both metrics into a single measure. It is calculated as:

$$F1 - score = \frac{2 \times (Precision \times Recall)}{Precision + Recall} \quad (27)$$

Comparison and results analysis

In this section, binary classification results are first presented to compare the proposed NIDS model with other methods, thereby evaluating its ability to distinguish between normal and attack traffic. Subsequently, a more complex multi-class classification experiment is conducted to assess the model's capacity to differentiate among various attack types.

Binary classification experiment

In the binary classification experiment, the proposed model is compared with several baseline NIDS methods: E-GraphSage²⁶, GCN³⁹, GAT⁴⁰, CNN-BILSTM⁴¹, and XMF-GNN⁴². To ensure a fair comparison, all graph neural network-based methods employed the same data preprocessing and graph construction procedures. Although the remaining machine learning and DL approaches utilized different data processing techniques, these methods were implemented according to their original configurations to achieve optimal performance on the dataset; therefore, they remain valid baselines for comparison. The experimental results, including accuracy, precision, recall, and F1-score, are presented in Table 6.

As shown in Table 6, the proposed model attains the highest performance on both the CIC-IDS2017 and InSDN datasets, achieving detection accuracies of 99.12% and 99.92%, respectively. Compared with the baseline GNN models, the proposed method achieves a 3% improvement in detection accuracy on the CIC-IDS2017 dataset and further outperforms the state-of-the-art XMF-GNN model with a 0.4% accuracy gain. The confusion matrix results for the proposed model on both datasets are presented in Fig. 4. As can be observed from the figure, only a minimal fraction of samples were misclassified or missed; notably, only 0.1% of attack samples were misclassified as normal traffic in the InSDN dataset, while all normal traffic samples were correctly classified.

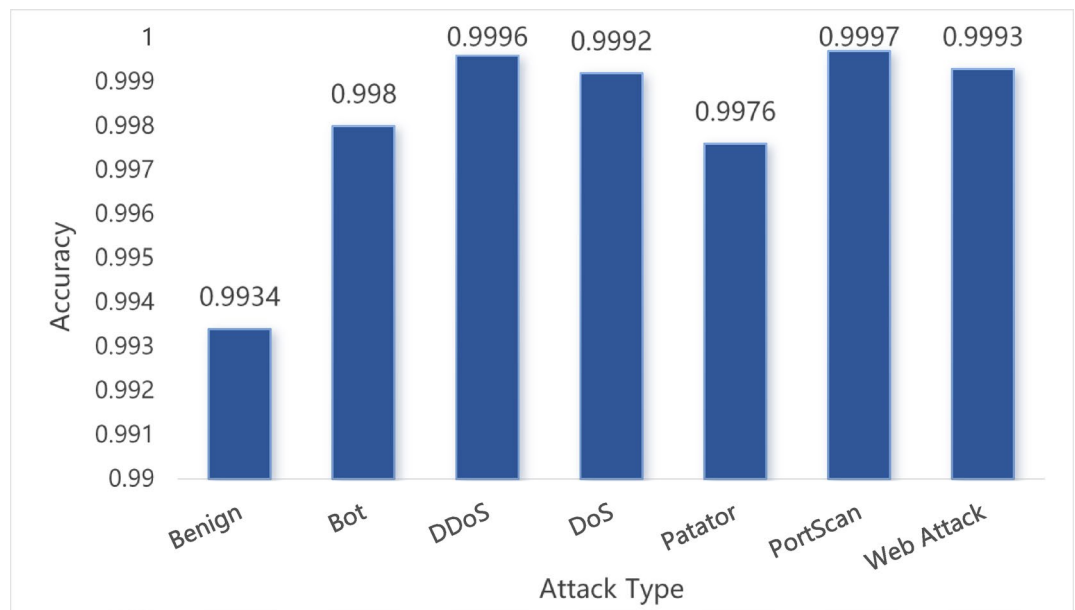


Fig. 5. Multi-class attack type detection accuracy for CIC-IDS2017.

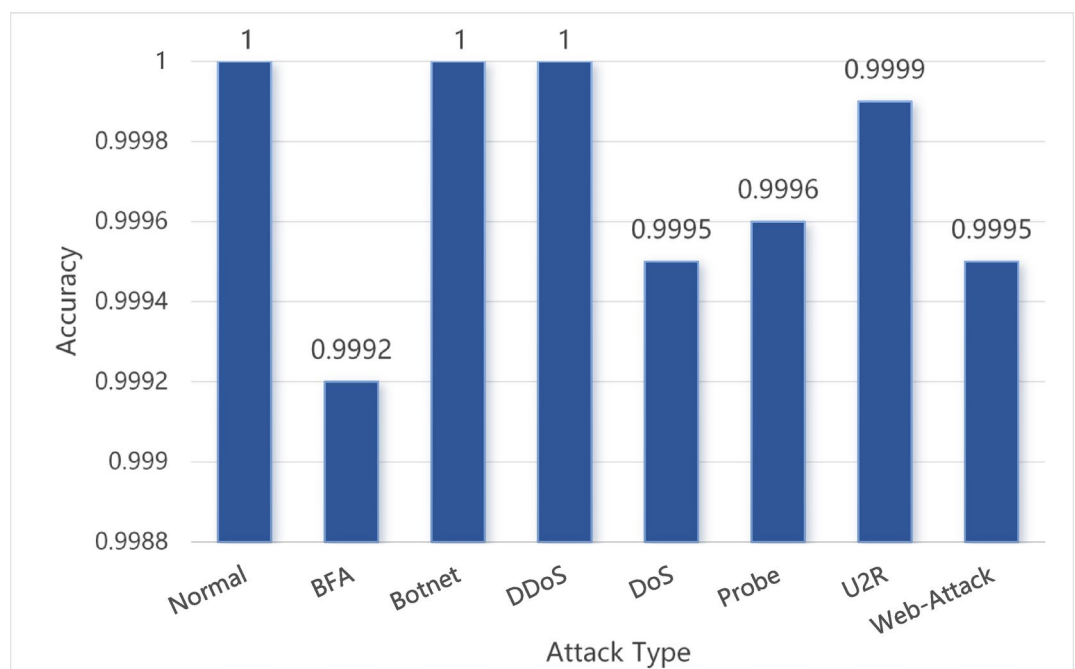


Fig. 6. Multi-class attack type detection accuracy for InSDN.

These experimental results demonstrate that the proposed method can effectively distinguish between normal and attack traffic, thereby enabling robust prevention of network attacks.

Multi-class experiment

Building upon the binary classification experiment, a more complex multi-class classification experiment is conducted to further evaluate the model's ability to distinguish among different attack types. Figures 5 and 6 present the multi-class experimental results for the CIC-IDS2017 and InSDN datasets, respectively. As can be observed from the figures, the proposed model effectively differentiates between attack types. In the CIC-IDS2017 experiment, the model achieves detection accuracies of 99.96% and 99.92% for DDoS and DoS attacks, respectively. This strong performance can be attributed to the design of the temporal detection module, as both

	E-GraphSage			GCN			GAT			DMSTG-AD		
	Precision	Recall	F1-score	Precision	Recall	F1-score	Precision	Recall	F1-score	Precision	Recall	F1-score
Benign	0.9570	0.9948	0.9755	0.9573	0.9871	0.9720	0.9635	0.9793	0.9713	0.9945	0.9975	0.9960
Bot	0.7704	0.6423	0.7005	0.7060	0.7380	0.7217	0.6578	0.6196	0.6381	0.9103	0.6650	0.7686
DDoS	0.9988	0.9732	0.9859	0.9977	0.9691	0.9832	0.9751	0.9431	0.9588	0.9955	0.9910	0.9932
DoS	0.9897	0.7040	0.8227	0.9863	0.7015	0.8199	0.9588	0.8083	0.8526	0.9941	0.9931	0.9936
Patator	0.9700	0.7930	0.8726	0.9811	0.5513	0.7059	0.9501	0.5534	0.6994	0.9669	0.9554	0.9611
PortScan	0.9921	0.7962	0.8834	0.9129	0.9782	0.9444	0.9058	0.9776	0.9403	0.9980	0.9945	0.9963
Web Attack	0.6941	0.8498	0.7641	0.5380	0.8565	0.6609	0.5054	0.8386	0.6307	0.9756	0.8946	0.9333
weighted avg	0.9599	0.9588	0.9567	0.9557	0.9532	0.9507	0.9531	0.9519	0.9504	0.9933	0.9934	0.9932

Table 7. Multi-class comparison experiment results for CIC-IDS2017.

	E-GraphSage			GCN			GAT			DMSTG-AD		
	Precision	Recall	F1-score	Precision	Recall	F1-score	Precision	Recall	F1-score	Precision	Recall	F1-score
Normal	0.9993	0.9920	0.9956	0.9988	0.9938	0.9963	0.9985	0.9925	0.9955	1.0000	1.0000	1.0000
BFA	0.0000	0.0000	0.0000	0.2564	0.0385	0.0669	0.0150	0.0885	0.0257	0.8388	0.9808	0.9043
Botnet	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.9000	1.0000	0.9474
DDoS	0.9985	0.9992	0.9989	0.9836	0.9991	0.9913	0.9831	0.9992	0.9911	1.0000	0.9999	0.9999
DoS	0.9610	0.9902	0.9754	0.8610	0.9921	0.9219	0.9794	0.7157	0.8270	0.9995	0.9971	0.9983
Probe	0.9897	0.9975	0.9936	0.9917	0.9186	0.9537	0.8738	0.9188	0.8957	0.9993	0.9993	0.9993
U2R	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.4615	1.0000	0.6316
Web-Attack	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.6250	0.1316	0.2174
weighted avg	0.9866	0.9911	0.9888	0.9695	0.9701	0.9685	0.9515	0.9334	0.9394	0.9988	0.9988	0.9987

Table 8. Multi-class comparison experiment results for InSDN.

DDoS and DoS attacks involve the rapid transmission of large volumes of attack traffic, enabling the model to effectively capture short-term burst patterns.

Additional multi-class classification experiments were conducted on both the CIC-IDS2017 and InSDN datasets using several baseline models, including E-GraphSAGE, GCN, and GAT. The detailed results are reported in Tables 7 and 8. As shown, the proposed DMSTG-AD model consistently outperforms the comparison methods across different attack categories. On the CIC-IDS2017 dataset, the overall weighted average F1 score of the proposed model is 99.32%, which is approximately 4% higher than the optimal baseline method. It demonstrates superior performance not only on benign traffic but also on diverse attack types, including DDoS, DoS, Botnet, and PortScan. Similarly, on the InSDN dataset, DMSTG-AD achieved the best overall results with a weighted average F1 score of 99.87%, an improvement of approximately 1% compared to the optimal baseline model. These results confirm that the proposed method provides more accurate and stable anomaly detection compared to existing approaches.

A comparative analysis of the experimental results achieved by the proposed model and existing state-of-the-art methods is presented in Fig. 7. As illustrated, the proposed model significantly outperforms the baseline models, achieving an approximate 1% improvement in overall detection accuracy on the CIC-IDS2017 dataset. Specifically, the proposed model attains a detection accuracy of 99.34%, whereas the comparison models—XMF-GNN, improved GraphSAGE⁴³, and XDLTDS⁴⁴—achieve 97.43%, 98.32%, and 98.57%, respectively.

On the InSDN dataset, as shown in Fig. 8, the detection accuracies for E-GraphSage, CNN-BILSTM, and GRAN⁴⁵ are 99.11%, 97.12%, and 97.1%, respectively. The proposed model once again achieved the highest performance, with a detection accuracy of 99.88%, an improvement of 0.77%.

These results demonstrate that the proposed model not only effectively distinguishes between normal and attack traffic but also accurately classifies specific attack types, thereby providing network administrators with reliable guidance for implementing targeted countermeasures against various network attacks.

Dynamic and multiscale analysis

To empirically verify that the proposed model effectively captures dynamic topological dependencies—a core contribution of DMSTG-AD—a qualitative case study is conducted on a DDoS attack from the CIC-IDS2017 dataset. The objective is to visualize how the adaptive adjacency matrix evolves as network traffic transitions from normal to attack and subsequently returns to normal.

A DDoS attack that occurred on 2017-07-07 from 03:55 to 04:00 (window 430 in the graph sequence) is selected for analysis. The preceding normal window (03:50–03:55, window 429) and the subsequent post-attack window (04:20–04:25, window 435) are used as baselines for comparison. After training, the learned adaptive adjacency matrix is extracted for each of the three windows, and the attention weights between four representative

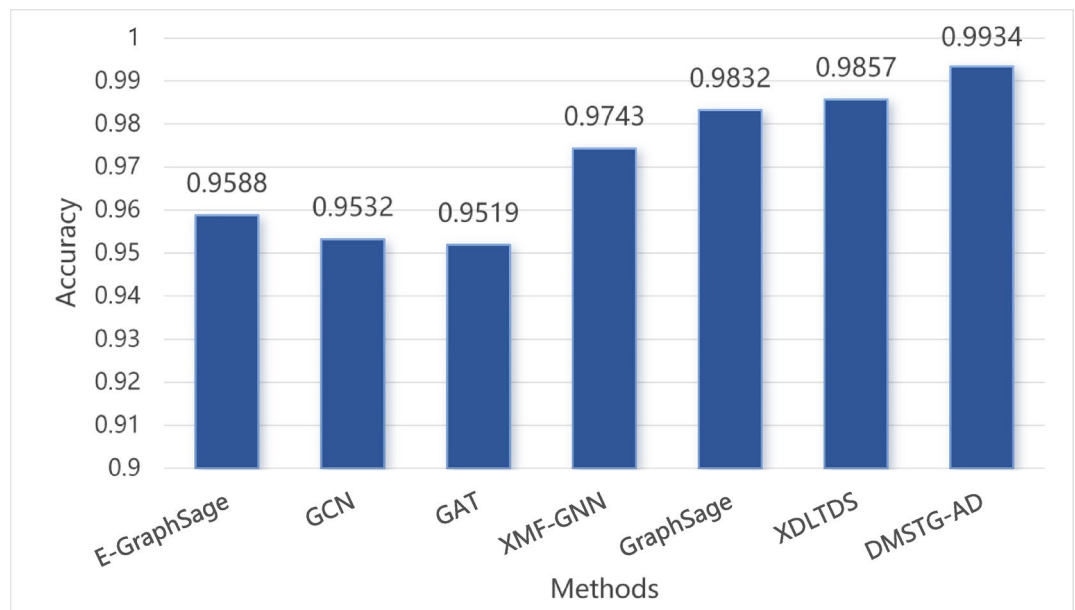


Fig. 7. Multi-classification detection accuracy comparison experiment for CIC-IDS2017.

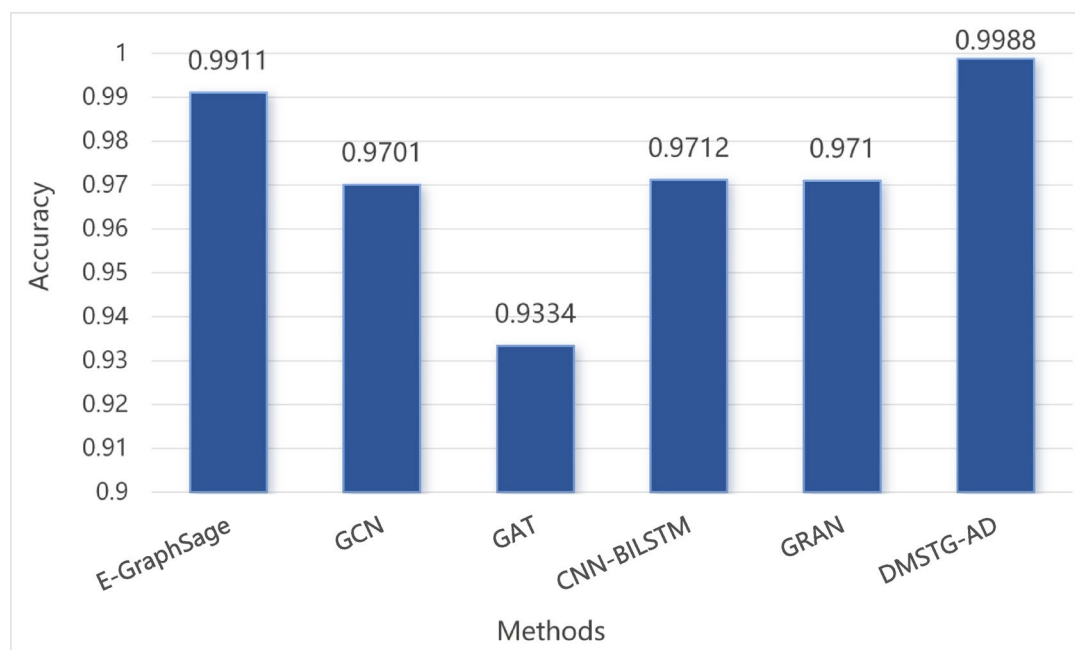


Fig. 8. Multi-classification detection accuracy comparison experiment for InSDN.

IP nodes are examined: 192.168.10.50 (an attacker node involved in the DDoS), 172.16.0.1 (another attacker node), 192.168.10.16 (a normal host), and 192.168.10.14 (another normal host). (Fig. 9)

Figure 10 presents heatmaps of the adaptive adjacency weights among these four nodes. In the pre-attack window, all pairwise weights are near zero. During the attack, the connection between the two attacker nodes sharply increases to 0.50, while normal-normal and attacker-normal weights remain negligible. After the attack subsides, the elevated weight between the attacker nodes returns to zero. This pattern is further illustrated in the line plot (Fig. 11), which tracks the average attention weight of attack-node pairs versus normal-node pairs across the three phases. Quantitative metrics corroborate this observation: the mean weight among attack nodes is 0.000 before the attack, 0.500 during the attack, and 0.000 after the attack; weights among normal nodes remain within the range of 0.014–0.016, with only a minor dip to 0.0033 during the attack window.

These findings provide direct evidence that the adaptive adjacency matrix is not static—it dynamically strengthens the connectivity between malicious nodes precisely when an attack is ongoing and attenuates it

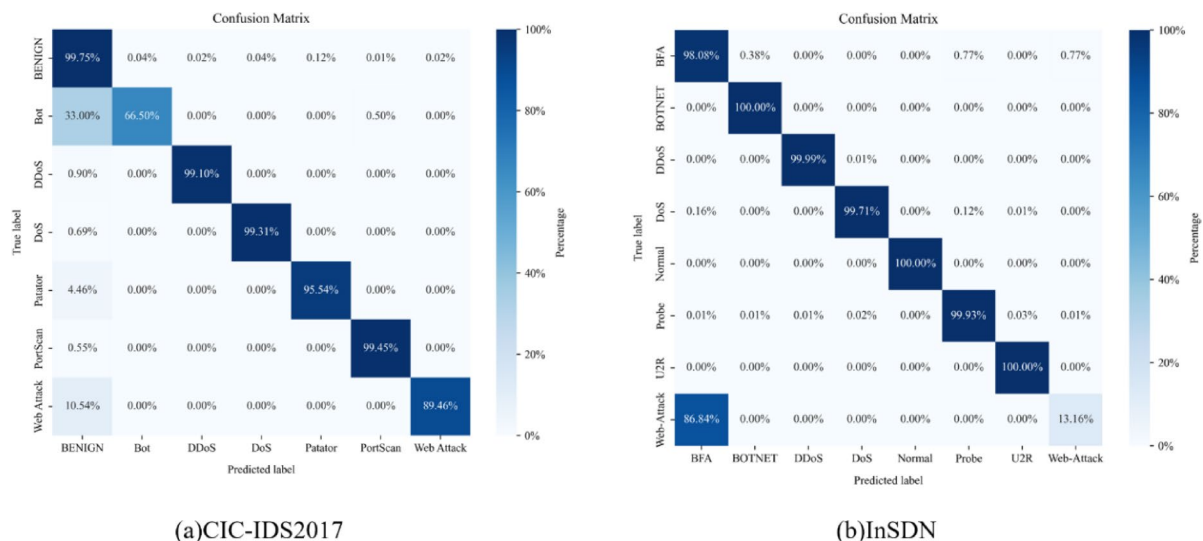


Fig. 9. Multi-class confusion matrix for CIC-IDS2017 and InSDN.

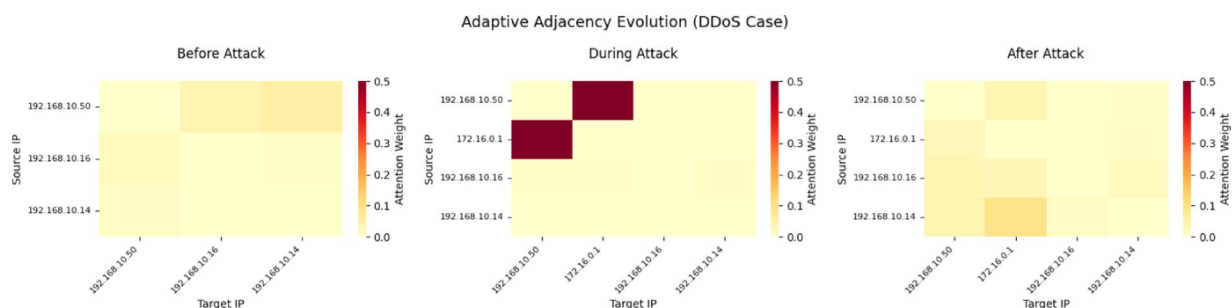


Fig. 10. The changes of adaptive adjacency matrix before, during, and after an attack.

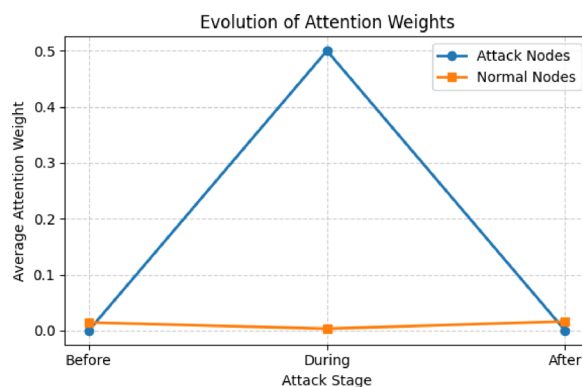


Fig. 11. Variations in attention weights of normal nodes versus attack nodes.

thereafter. The model thus successfully learns to rewire the graph topology in accordance with the current traffic state, thereby fulfilling the “dynamic” claim central to DMSTG-AD.

To further validate the effectiveness of the proposed multi-scale temporal modeling mechanism, the activation patterns of temporal components with different receptive fields are analyzed during the same DDoS attack event. The results are illustrated in Fig. 12.

As shown in the figure, distinct temporal components exhibit clearly differentiated behaviors. The TCN with dilation rate $d = 1$ demonstrates sharp and highly localized activation peaks, which closely align with abrupt traffic bursts during the attack window. This indicates that small receptive fields are particularly sensitive to short-term fluctuations and micro-burst behaviors. The TCN with dilation rate $d = 3$ produces smoother

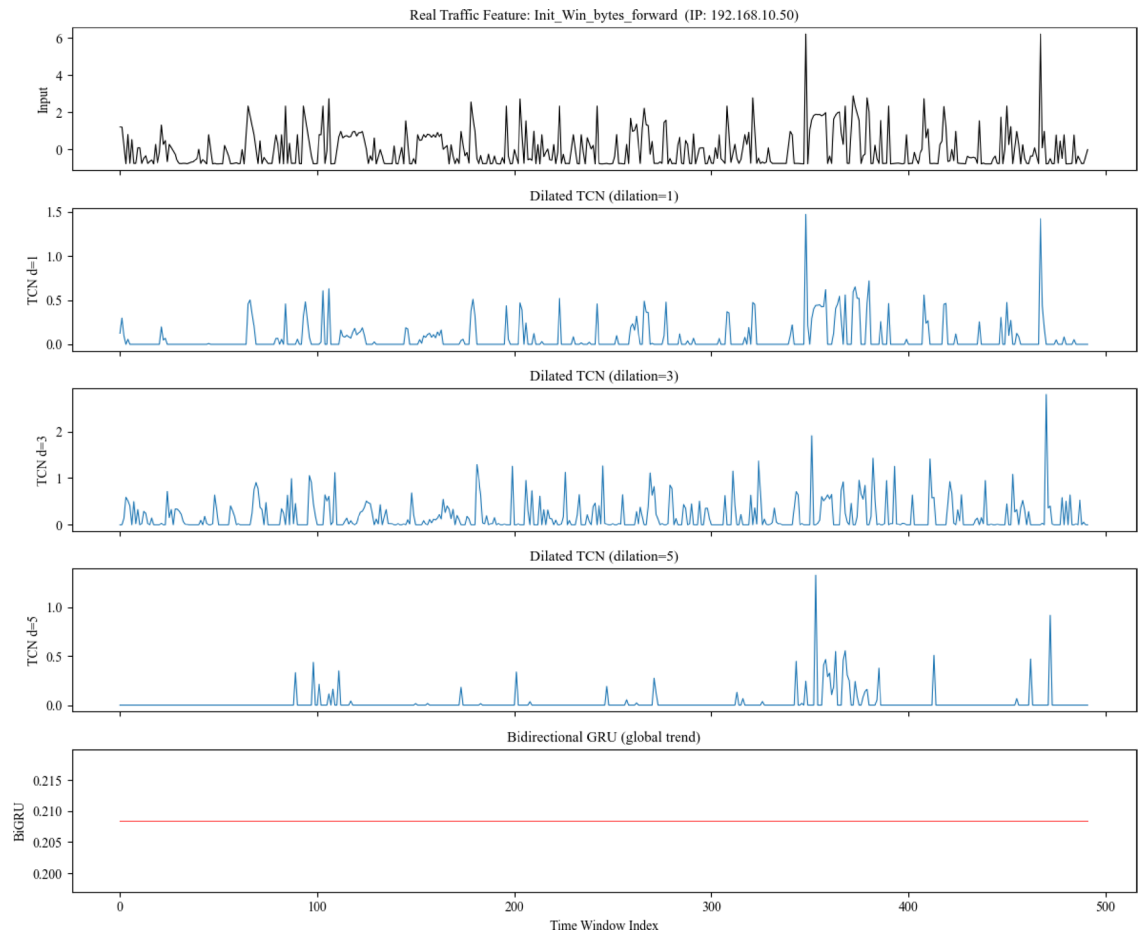


Fig. 12. Comparative visualization of temporal feature responses across different receptive fields during an attack window.

and broader activation regions. Compared to $d = 1$, it captures mid-range temporal correlations rather than isolated spikes, reflecting its capacity to model intermediate-scale dependencies. The TCN with dilation rate $d = 5$ exhibits sparse yet sustained activation primarily within the core attack interval, effectively suppressing transient noise outside the attack window. This suggests that larger receptive fields are better suited for modeling long-term structural changes induced by persistent attacks. The BiGRU output remains relatively smooth and trend-oriented across the entire time span. Unlike the TCN layers, it does not respond strongly to local bursts but instead captures the overall temporal evolution of traffic patterns.

These results demonstrate that different temporal components specialize in modeling distinct time scales. Rather than redundantly learning similar features, the small-dilation TCN captures short-term burst dynamics, the larger-dilation TCNs capture long-term structural attack patterns, and the BiGRU models global temporal trends. Consequently, the proposed temporal module achieves genuine multi-scale temporal representation learning, thereby substantiating the “multi-scale” design claim central to DMSTG-AD.

Representational analysis

To further validate that the representations learned by DMSTG-AD are more discriminative beyond what is reflected in classification accuracy, an additional representational analysis experiment is conducted on the InSDN dataset. Specifically, node embeddings are extracted from the final hidden layer of the trained models using the InSDN test set, and t-SNE is applied for dimensionality reduction and visualization. For comparison, the same procedure is performed for a static GCN baseline model. The resulting two-dimensional projections are illustrated in Fig. 13.

As shown in Fig. 13, clear differences can be observed between the two embedding spaces. In the embedding space produced by the proposed DMSTG-AD model, samples from different traffic categories form compact and well-structured clusters. Normal traffic is clearly separated from various attack types, and most attack categories exhibit tight intra-class distributions with distinct inter-class boundaries. The resulting clusters are visually coherent, indicating that the learned representation space preserves strong semantic structure and discriminative information.

In contrast, the embeddings generated by the static GCN baseline exhibit significant inter-class overlap. Several attack categories are intermixed with normal traffic samples, and the cluster boundaries appear less

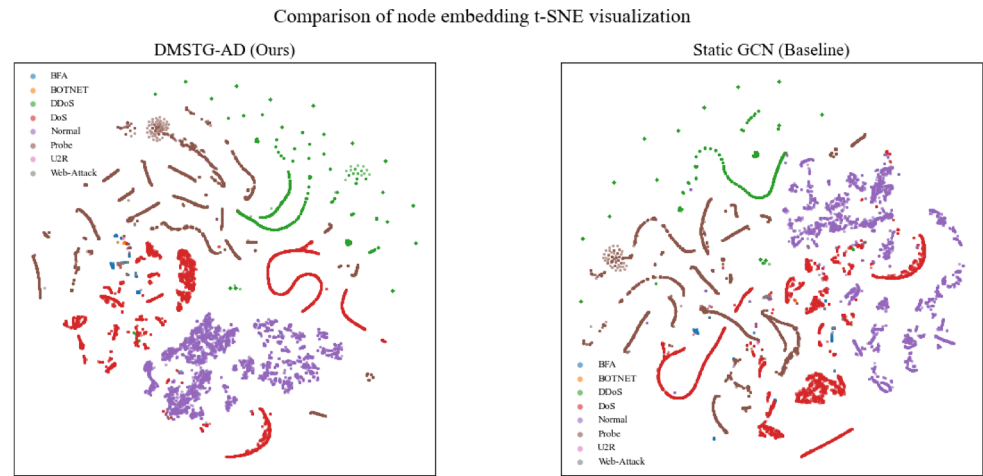


Fig. 13. Visualization results of node embedding using t-SNE on the InSDN dataset.

Model	Model Size(MB)	Average Sample Time(ms)	Throughput(Sample/s)	Total Inference Time(s)
E-GraphSAGE:	0.30	0.0016	3103.00	0.16
GAT	1.20	0.0034	1543.46	0.32
GCN	0.15	0.0028	2143.22	0.23
DMSTG-AD	2.35	0.11	55.78	8.75

Table 9. Efficiency comparison of DMSTG-AD and baseline models.

distinct. The overall structure is more fragmented and less separable, suggesting that static graph modeling fails to adequately capture the dynamic dependencies and temporal variations inherent in network traffic.

Efficiency and scalability

To assess the practical deployment feasibility of DMSTG-AD in real-world NIDS, a comprehensive analysis of its efficiency and scalability was conducted in comparison with baseline models. Key evaluation metrics include model size, average sample processing time (i.e., inference latency), throughput, and total inference time, all of which are summarized in Table 9.

As shown in Table 9, DMSTG-AD has a model size of 2.35 MB. While this is larger than the lightweight GCN (0.15 MB) and E-GraphSAGE (0.30 MB), it remains sufficiently compact for storage and deployment in resource-constrained environments. In terms of inference speed, DMSTG-AD achieves an average sample processing time of 0.11 ms. Although this is slower than the simpler graph-based models—which benefit from shallower architectures and fewer parameters—it still demonstrates efficient inference suitable for real-time processing in most network traffic scenarios.

The lower throughput (55.78 samples/s) compared to baselines reflects the computational complexity inherent in modeling dynamic spatial–temporal dependencies. However, the total inference time of 8.75 s remains acceptable for batch processing in offline analysis or near-real-time applications. These results indicate that while DMSTG-AD introduces moderate overhead due to its sophisticated architecture, it strikes a reasonable balance between model complexity and practical deployability. Future work may explore optimization strategies such as graph pruning and model quantization to further enhance scalability to larger network topologies.

Ablation study

To evaluate the effectiveness of the proposed STCA mechanism, a detailed ablation study was conducted, with results for different configurations presented in Fig. 14. Three ablation experiments were performed by removing individual modules from the complete DMSTG-AD model while retaining all other modules unchanged, thereby enabling the contribution of each component to be isolated and evaluated.

The first ablation experiment removed the temporal feature extraction module (Wo_temporal). In this configuration, the model retained the three-layer GCN with edge-aware mechanisms to capture spatial topological information, but did not model temporal evolution across different graphs and nodes. The STCA fusion was still applied to the spatial features, though without temporal input its effectiveness was limited. As shown in Fig. 14, removing the temporal module caused the accuracy to drop from 99.34% to 97.96% on CIC-IDS2017 and from 99.88% to 99.45% on InSDN, corresponding to decreases of 1.38% and 0.43%, respectively.

The second ablation experiment removed the spatial feature extraction module (Wo_spatial). Here, the temporal module (TCN and BiGRU) was retained to extract temporal traffic patterns, but the GCN and edge-aware modules were omitted. The STCA mechanism was still present, though it now operated on temporal

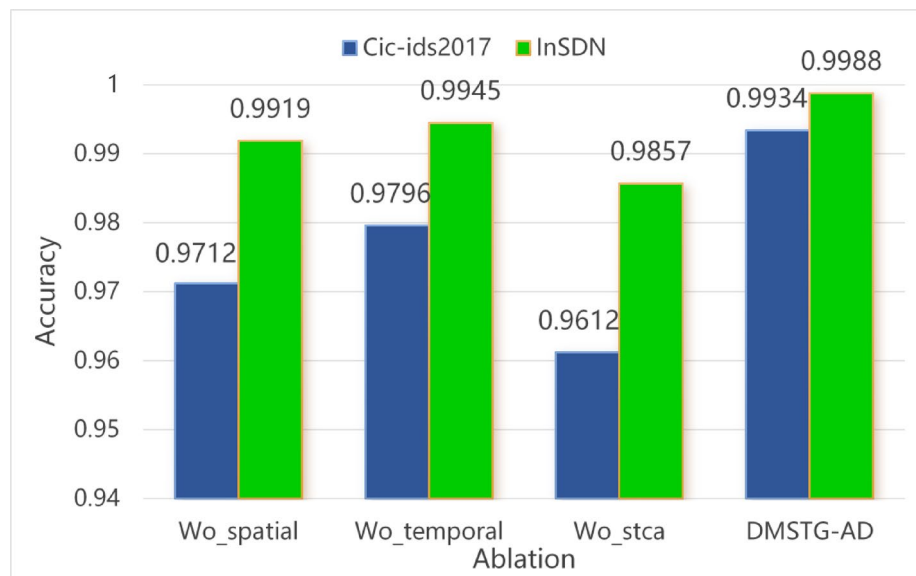


Fig. 14. Ablation study results.

features only. As illustrated in Fig. 14, the removal of spatial information led to a substantial performance decline: accuracy fell to 97.12% on CIC-IDS2017 and 99.19% on InSDN, representing reductions of 2.22% and 0.69% relative to the full model. These larger drops compared to the temporal ablation highlight the critical role of spatial topological structure in network attack detection.

The third ablation experiment removed the STCA fusion module (Wo_stca) while retaining both the spatial and temporal feature extractors. Without this fusion mechanism, the model simply concatenated spatial and temporal features without learning their cross-dependencies. As shown in Fig. 14, this resulted in the most severe performance degradation: accuracy dropped to 96.12% on CIC-IDS2017 and 98.57% on InSDN, i.e., decreases of 3.22% and 1.31% compared to the complete model. These results demonstrate that the cross-attention fusion is indispensable for effectively combining spatial and temporal information.

Comparing the magnitude of accuracy drops across the three ablation settings reveals a clear ranking of module contributions. On both datasets, the removal of the spatio-temporal cross-attention mechanism caused the largest performance loss, indicating that this fusion module is the most critical component. The spatial module contributed the next largest improvement, followed by the temporal module. This ranking underscores that while temporal patterns are beneficial, the spatial topology of network traffic and, especially, the synergistic fusion of spatial and temporal features are the key factors driving the high detection accuracy of DMSTG-AD.

Conclusion

In this paper, a new paradigm for intrusion detection has been established by demonstrating that joint dynamic spatio-temporal learning is not merely advantageous but essential for modeling modern network threats. Building upon a critical examination of existing GNN-based NIDS, a fragmentation was identified among static-topology, node-centric temporal, and graph-level temporal modeling approaches. The proposed DMSTG-AD framework serves as a synthesis of these perspectives, grounded in the theoretical insight that attacks simultaneously manifest as node-level behavioral anomalies and graph-level topological rewirings. This insight is realized through a principled architecture comprising adaptive adjacency matrices, multi-scale temporal convolutions, and spatio-temporal cross-attention. The empirical results—demonstrating consistent improvements across diverse attack scenarios on multiple benchmark datasets—validate that unifying these dimensions provides a more resilient and fine-grained representation of network dynamics. This work lays the foundation for future NIDS to be designed not as static classifiers, but as holistic observers of evolving digital ecosystems.

Future work will explore two primary directions: (1) investigating the deployment, optimization, and interpretability of the model in large-scale real-time networks to improve inference efficiency and scalability; and (2) examining its integration with federated learning, adversarial training, and other advanced techniques to enhance robustness in distributed and adversarial settings. The proposed framework is expected to contribute not only to advancing intrusion detection research but also to supporting the development of more secure and intelligent next-generation network architectures.

Data availability

The data supporting this study are available when reasonably requested from the corresponding author.

Received: 26 October 2025; Accepted: 11 March 2026

Published online: 22 March 2026

References

1. Tang, Y., Wang, W.-Z., Wang, W.-Y. & Xu, B.-W. Design and implementation of the interconnected architecture of SDN and traditional IP network. *Computer Eng. Sci.* **39**, 2210 (2017).
2. Kreutz, D. et al. Software-defined networking: A comprehensive survey. *Proc. IEEE* **103**, 14–76 (2014).
3. Yoon, C. et al. Flow wars: Systemizing the attack surface and defenses in software-defined networks. *IEEE/ACM Trans. Networking* **25**, 3514–3530 (2017).
4. Taheri, R., Ahmed, H. & Arslan, E. Deep learning for the security of software-defined networks: A review. *Cluster Comput.* **26**, 3089–3112 (2023).
5. Chica, J. C. C., Imbach, J. C. & Vega, J. F. B. Security in SDN: A comprehensive survey. *J. Netw. Comput. Appl.* **159**, 102595 (2020).
6. Balakrishnan, S., Ivy, B. P. U. & Ilango, S. S. A novel and secured intrusion detection system for wireless sensor networks using identity based online/offline signature. *ARPJ. Eng. Appl. Sci.* **13**, 8544–8547 (2018).
7. Saathvika, S., Accamma, B. & Santhosh Kumar, B. in *2024 Control Instrumentation System Conference (CISCON)*. 1–7 (IEEE).
8. Mishra, P., Varadharajan, V., Tupakula, U. & Pilli, E. S. A detailed investigation and analysis of using machine learning techniques for intrusion detection. *IEEE Commun. Surv. Tut.* **21**, 686–728 (2018).
9. Gu, J. & Lu, S. An effective intrusion detection approach using SVM with naïve Bayes feature embedding. *Comput. Secur.* **103**, 102158 (2021).
10. Ahuja, N., Singal, G., Mukhopadhyay, D. & Kumar, N. Automated DDOS attack detection in software defined networking. *J. Netw. Comput. Appl.* **187**, 103108 (2021).
11. Wu, K., Chen, Z. & Li, W. A novel intrusion detection model for a massive network using convolutional neural networks. *IEEE Access* **6**, 50850–50859 (2018).
12. Malik, J. et al. Hybrid deep learning: An efficient reconnaissance and surveillance detection mechanism in SDN. *IEEE Access* **8**, 134695–134706 (2020).
13. Bilot, T., El Madhoun, N., Al Agha, K. & Zouaoui, A. Graph neural networks for intrusion detection: A survey. *IEEE Access* **11**, 49114–49139 (2023).
14. Caville, E., Lo, W. W., Layeghy, S. & Portmann, M. Anomal-E: A self-supervised network intrusion detection system based on graph neural networks. *Knowl. Based Syst.* **258**, 110030 (2022).
15. Altaf, T., Wang, X., Ni, W., Liu, R. P. & Braun, R. NE-GConv: A lightweight node edge graph convolutional network for intrusion detection. *Comput. Secur.* **130**, 103285 (2023).
16. Liu, H. & Lang, B. Machine learning and deep learning methods for intrusion detection systems: A survey. *Appl. Sci. (Basel)* **9**, 4396 (2019).
17. ElSayed, M. S., Le-Khac, N.-A., Albahar, M. A. & Jurcut, A. A novel hybrid model for intrusion detection systems in SDNs based on CNN and a new regularization technique. *J. Netw. Comput. Appl.* **191**, 103160 (2021).
18. Ravi, V., Chaganti, R. & Alazab, M. Recurrent deep learning-based feature fusion ensemble meta-classifier approach for intelligent network intrusion detection system. *Comput. Electr. Eng.* **102**, 108156 (2022).
19. Assis, M. V., Carvalho, L. F., Lloret, J. & Proença, M. L. Jr. A GRU deep learning system against attacks in software defined networks. *J. Netw. Comput. Appl.* **177**, 102942 (2021).
20. Elsayed, R. A., Hamada, R. A., Abdalla, M. I. & Elsaid, S. A. Securing IoT and SDN systems using deep-learning based automatic intrusion detection. *Ain Shams Eng. J.* **14**, 102211 (2023).
21. Nguyen, T. G., Phan, T. V., Hoang, D. T., Nguyen, T. N. & So-In, C. Federated deep reinforcement learning for traffic monitoring in SDN-based IoT networks. *IEEE Trans Cognit Commun Netw* **7**, 1048–1065 (2021).
22. Wang, H. & Li, W. DDosTC: A transformer-based network attack detection hybrid mechanism in SDN. *Sensors* **21**, 5047 (2021).
23. Behera, A., Sahoo, K. S., Mishra, T. K. & Bhuyan, M. A combination learning framework to uncover cyber attacks in IoT networks. *Internet of Things* **28**, 101395 (2024).
24. Guo, Y. et al. Traffic management in IoT backbone networks using GNN and MAB with SDN orchestration. *Sensors* **23**, 7091 (2023).
25. Zhou, X. et al. Hierarchical adversarial attacks against graph-neural-network-based IoT network intrusion detection system. *IEEE Internet Things J.* **9**, 9310–9319 (2021).
26. Lo, W. W., Layeghy, S., Sarhan, M., Gallagher, M. & Portmann, M. E-graphsage: A graph neural network based intrusion detection system for iot. *arXiv preprint arXiv:2103.16329* (2021).
27. Lan, J. et al. E-minBatch GraphSAGE: An Industrial Internet Attack Detection Model. *Security and Communication Networks* **2022**, 5363764 (2022).
28. Deng, P. & Huang, Y. Edge-featured multi-hop attention graph neural network for intrusion detection system. *Comput. Secur.* **148**, 104132 (2025).
29. Wang, Y. et al. N-STGAT: Spatio-temporal graph neural network based network intrusion detection for near-earth remote sensing. *Remote Sensing* **15**, 3611 (2023).
30. Duan, G., Lv, H., Wang, H., Feng, G. & Li, X. Practical cyber attack detection with continuous temporal graph in dynamic network system. *IEEE Trans. Inf. Forensics Secur.* **19**, 4851–4864 (2024).
31. Duan, G., Lv, H., Wang, H. & Feng, G. Application of a dynamic line graph neural network for intrusion detection with semisupervised learning. *IEEE Trans. Inf. Forensics Secur.* **18**, 699–714 (2022).
32. Xu, D., Ruan, C., Korpeoglu, E., Kumar, S. & Achan, K. Inductive representation learning on temporal graphs. *arXiv preprint arXiv:2002.07962* (2020).
33. Benzekki, K., El Fergougui, A. & Elbelrhiti Elalaoui, A. Software-defined networking (SDN): A survey. *Secur. Commun. Netw.* **9**, 5803–5833 (2016).
34. Chen, Q. et al. An adaptive adjacency matrix-based graph convolutional recurrent network for air quality prediction. *Sci. Rep.* **14**, 4408 (2024).
35. Dey, R. & Salem, F. M. in *2017 IEEE 60th international midwest symposium on circuits and systems (MWSCAS)*. 1597–1600 (IEEE).
36. Lea, C., Flynn, M. D., Vidal, R., Reiter, A. & Hager, G. D. in *proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 156–165.
37. Sharafaldin, I., Lashkari, A. H. & Ghorbani, A. A. Toward generating a new intrusion detection dataset and intrusion traffic characterization. *ICISSp* **1**, 108–116 (2018).
38. Elsayed, M. S., Le-Khac, N.-A. & Jurcut, A. D. InSDN: A novel SDN intrusion dataset. *IEEE Access* **8**, 165263–165284 (2020).
39. Kipf, T. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907* (2016).
40. Veličković, P. et al. Graph attention networks. *arXiv preprint arXiv:1710.10903* (2017).
41. Said, R. B., Sabir, Z. & Askerzade, I. CNN-BiLSTM: A hybrid deep learning approach for network intrusion detection system in software-defined networking with hybrid feature selection. *IEEE Access* **11**, 138732–138747 (2023).
42. Ma, Z., Liu, Y., Chen, Y., Liu, Z. & Li, Y. XMF-GNN: A cross-modality dynamic fusion heterogeneous graph neural network for network intrusion detection. *Neurocomputing* <https://doi.org/10.1016/j.neucom.2025.131285> (2025).
43. Le, H.-D. & Park, M. Enhancing multi-class attack detection in graph neural network through feature rearrangement. *Electronics* **13**, 2404 (2024).
44. Shoukat, S., Gao, T., Javeed, D., Saeed, M. S. & Adil, M. Trust my IDS: An explainable AI integrated deep learning-based transparent threat detection system for industrial networks. *Comput. Secur.* **149**, 104191 (2025).

45. Zhang, Y., Jue, C., Liu, W. & Ma, Y. GRAN: A SDN intrusion detection model based on graph attention network and residual learning. *Comput. J.* **68**, 241–260 (2025).

Author contributions

J.Z.: Conceptualization, Methodology, Software, Validation, Formal analysis, Writing—original draft preparation. D.Z.: Validation, Resources. Q.H.: Writing—review and editing, Formal analysis. M.L.: Writing—review and editing. Y. Y.: Writing—review and editing.

Funding

This work is sponsored by the National Natural Science Foundation of China (NO.52578067).

Declarations

Competing interests

The authors declare no competing interests.

Additional information

Correspondence and requests for materials should be addressed to D.Z.

Reprints and permissions information is available at www.nature.com/reprints.

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

© The Author(s) 2026